



UNIVERSITY OF PISA

Department of Information Engineering

Master's degree in Artificial Intelligence and Data Engineering

Master's degree in Computer Engineering

Large-Scale and Multi-Structured Databases

RiskDeV

Work Group:
Tommaso Dei
Filippo Gambelli
Andrea Zanin

ACADEMIC YEAR 2025/2026

Contents

1	Introduction	4
2	Design Overview	5
2.1	Main Actors	5
2.2	Requirements	5
2.2.1	Functional Requirements	5
2.2.2	Non-Functional Requirements	7
2.3	UML Class Diagram	8
2.4	User Interface Mockups	8
2.4.1	Guest (Unregistered User)	9
2.4.2	Registered User	10
2.4.3	Administrator	11
3	Dataset Creation	12
3.1	Package Metadata and Vulnerabilities	12
3.2	Synthetic Projects and Users	12
3.3	Dataset Size	13
4	Data Modeling and Structure	14
4.1	MongoDB	14
4.1.1	Package Collection	15
4.1.2	Project Collection	16
4.1.3	User Collection	16
4.1.4	Vulnerability Collection	17
4.1.5	Dead Letter Queue (DLQ) Collection	18
4.1.6	Document Embedding and Linking	18
4.2	Neo4j	19
4.2.1	Nodes	19
4.2.2	Relationships	19
4.2.3	Data Population Process	19
5	Implementation	22
5.1	Layered Architecture and Design Patterns	22
5.2	Logging Configuration	23
5.3	Project Structure and Package Organization	24
5.3.1	it.unipi.riskDeV.config	24
5.3.2	it.unipi.riskDeV.controller	24
5.3.3	it.unipi.riskDeV.service	26
5.3.4	it.unipi.riskDeV.repository	27
5.3.5	it.unipi.riskDeV.model	28

5.3.6	it.unipi.riskDeV.DAO	29
5.3.7	it.unipi.riskDeV.DTO	29
5.3.8	it.unipi.riskDeV.security	30
5.3.9	it.unipi.riskDeV.results	30
5.3.10	it.unipi.riskDeV.util	31
5.3.11	it.unipi.riskDeV.async	32
6	Relevant Queries	34
6.1	CRUD Operations	34
6.1.1	Vulnerability Controller	34
6.1.2	Projects Controller	36
6.1.3	Administrator Controller	41
6.1.4	Package Controller	43
6.1.5	Authentication Controller	48
6.1.6	User Controller	49
6.2	MongoDB Aggregation	52
6.2.1	Packages Risk Score Analysis	52
6.2.2	Monthly Vulnerability Trend Analysis	54
6.2.3	Critical Threat Identification	57
6.2.4	Top Used Packages Analysis	60
6.2.5	Trending Packages Analysis (Last Month)	62
6.2.6	Active Contributors Analysis	64
7	Database Architecture	67
7.1	Distributed Data Strategy	67
7.2	Mongo Replica Set	67
7.3	Infrastructure Topology	68
7.4	Eventual Consistency Across MongoDB and Neo4j	69
7.5	Sharding Strategy	72
7.5.1	Proposed Sharding Design	72
8	System Performance Evaluation	73
8.1	Indexes	73
8.1.1	MongoDB	73
8.1.2	Neo4j	80
8.2	Read Preference Performance Comparison	82
9	Use of Generative AI Tools	84

1 Introduction

RiskDev is a platform for exploring the Python package ecosystem with a strong focus on security. It collects and organizes package metadata across many versions and shows how packages depend on each other. This helps users understand both direct and indirect relationships between packages.

The main goal of RiskDev is to make security analysis faster and easier. Users can search and run queries on Python packages and on the vulnerabilities that affect them, with special attention to risks that are not immediately visible. A package may appear safe on its own, but it can still be risky if one of its dependencies, or a dependency further down the chain, is affected by a Common Vulnerabilities and Exposures (CVE) entry. RiskDev helps users identify these inherited vulnerabilities and choose package versions that reduce overall risk.

The fast growth of the Python ecosystem and the frequent release of new package versions require a scalable and efficient data management approach. To ensure low response times and reliable data, RiskDev uses a *polyglot persistence* architecture based on two NoSQL databases that work together. MongoDB stores large volumes of unstructured metadata about packages and their versions. Neo4j stores the dependency graph and allows efficient traversal of complex dependency chains and vulnerability paths.

RiskDev includes features for registered users. Users can create *projects*, invite collaborators, and manage a shared set of packages. A project represents the collection of Python packages that a team uses, or plans to use, in a real system. Project members can analyze their dependency tree, detect vulnerabilities that affect their stack, and identify versions where vulnerabilities have been fixed. This helps improve the security of their applications before and during development.

In addition to registered users, the system provides a higher level of authentication for system administrators. Administrators can run advanced and complex queries not only on packages and vulnerabilities, but also on how the system itself is used. This includes analysis of user activity, usage patterns and system performance. Administrators can also study security trends, such as the evolution of vulnerabilities over time and the behavior of packages whose vulnerability metrics do not improve even when new versions are released.

All source code is available at: <https://github.com/FilippoGambelli/RiskDev>

2 Design Overview

This chapter presents the overall design of the RiskDev platform and describes its main architectural and technical components.

The description of the system starts from a high-level view of the main actors who interact with the platform and the roles they play. It then defines the functional and non-functional requirements that guide the behavior, performance and security of the system.

Finally, this chapter introduces the core structural models, including the UML class diagram and provides conceptual user interface mockups to show how the system can be used from the perspective of different types of users.

2.1 Main Actors

The system is designed to interact with three primary actors, each characterized by distinct goals and privilege levels within the platform.

Unregistered Users (Guest) This actor represents any visitor who accesses the platform without creating an account or logging in. Guests can search for Python packages, view package metadata, and inspect both direct and indirect vulnerabilities. Guests can also register for an account to access advanced features, but they cannot create or manage projects.

Registered Users This actor represents a user who has created an account and logged into the system. Registered Users can perform all actions available to Guests. In addition, they can create and manage *projects*, which are containers for Python packages and their versions. Within a project, users can run queries on their packages to analyze their working environment, detect vulnerabilities and check if safer or newer versions are available. Project owners can also share their projects with other Registered Users to support collaboration.

Administrator The Administrator represents a system-level user responsible for managing and monitoring the entire platform. Administrators can perform all actions available to Registered Users. In addition, they can run advanced and complex queries on packages, vulnerabilities, and system usage. This includes analyzing security trends, monitoring active users, and ensuring the overall health and correct operation of the RiskDev system.

2.2 Requirements

2.2.1 Functional Requirements

This section defines the functional behavior that the system must support for each user category.

Unregistered User (Guest)

The system must allow a Guest to perform the following actions:

- The Guest must be able to search for a Python package by specifying only the package name.
 - The system must return general metadata and information about the latest available version of the specified package including the list of direct (first-level) dependencies and vulnerabilities that directly affect the package version.

- The Guest must be able to specify both a package name and a package version to retrieve detailed metadata for that version.
 - The system must return general metadata and information about the latest available version of the specified package including the list of direct (first-level) dependencies and vulnerabilities that directly affect the package version.
- The Guest must be able to see the full list of indirect (transitive) dependencies for the specified python package.
- The Guest must be able to see the vulnerabilities inherited from all dependencies of the specified python package.
- The Guest must be able to select a vulnerability and view detailed information, including description, publication date, and severity metrics.
- The Guest must be able to register a new account.
- The Guest must be able to log into the system after registration.

Registered User

A Registered User must be able to perform all actions available to a Guest. In addition, the system must allow a Registered User to perform the following actions:

- The Registered User must be able to create new projects that act as containers for Python packages and their versions.
- The Registered User must be able to modify project properties, including the project name and description.
- The Registered User must be able to delete projects they own.
- The Registered User must be able to specify the Python version used for each project.
- The Registered User must be able to add and remove packages within projects they own or collaborate on.
- The Registered User must be able to invite and remove collaborators for projects they own.
- The Registered User must be able to see, for each project they own or collaborate on, aggregated statistics, including:
 - The total number of packages in the project.
 - The total number of vulnerabilities in the project.
 - The overall project risk score calculated from vulnerability metrics.
 - The risk score and number of vulnerabilities for each package.
- The Registered User must be able to select a package within a project and view its metadata, direct dependencies and vulnerabilities.
- The Registered User must be able to log out of the system.
- The Registered User must be able to permanently delete their account.

Administrator

An Administrator must be able to perform all actions available to a Registered User. In addition, the system must allow an Administrator to perform the following actions:

- The Administrator must be able to view vulnerability trends over time.
- The Administrator must be able to retrieve statistics about system usage, including active users and the most active projects.
- The Administrator must be able to retrieve a list of packages ranked by PageRank score to identify globally influential packages in the dependency network.
- The Administrator must be able to retrieve a list of packages ranked by Degree Centrality to identify core packages based on the number of direct dependents.
- The Administrator must be able to retrieve packages grouped into risk score intervals (e.g., 0–2, 2–4, 4–6, 6–8, 8–10) to analyze the overall risk distribution of the ecosystem.
- The Administrator must be able to retrieve a list of the most frequently used dependency packages to identify adoption trends.
- The Administrator must be able to retrieve a list of vulnerabilities that are network-accessible, require low complexity to exploit, and have high or critical severity.
- The Administrator must be able to manage user roles by promoting Registered Users to Administrators and revoking Administrator privileges.

2.2.2 Non-Functional Requirements

This section defines the quality attributes, performance constraints, and technical standards that the system must satisfy to ensure reliability, efficiency, and maintainability.

- The system must support multiple concurrent user sessions and API requests without significant degradation in response time or throughput.
- The system must be designed to scale in order to handle growth in both user traffic and data volume.
- The system must ensure high availability and minimize downtime to provide continuous access to package data and analysis tools, even if some data is occasionally outdated.
- The system must be developed using Object-Oriented Programming principles to promote modularity, maintainability and code reuse.
- The system must expose its core functionalities through RESTful APIs to ensure interoperability with client applications.
- The system must implement a hybrid database architecture based on a document-oriented database (MongoDB) and a graph database (Neo4j).
- The system must be able to tolerate partial failures and handle exceptions gracefully without causing a complete system outage.
- The system must enforce role-based access control to protect sensitive and administrative functionalities based on user roles (Guest, Registered User and Administrator).

- The system must securely store user credentials using cryptographic hashing and must never store passwords in plain text.
- The system must return clear and descriptive error messages, mapped to appropriate HTTP status codes, to support effective debugging and user understanding.
- The system must be optimized for speed, ensuring quick responses to queries, filtered searches, dependency traversals, and vulnerability analyses.
- The system must execute complex graph traversals and large data aggregations, such as dependency trees and vulnerability lists, with low latency to ensure a responsive user experience.

2.3 UML Class Diagram

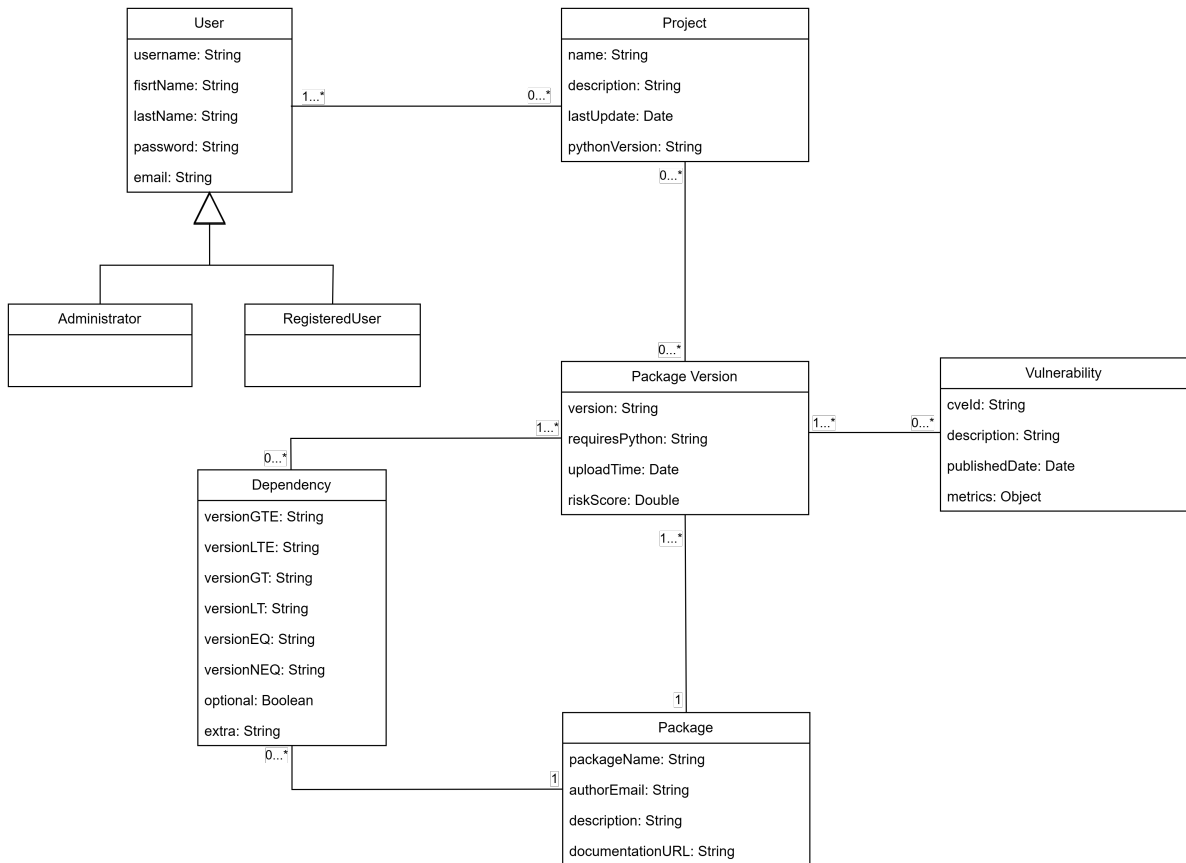


Figure 2.1: UML Class Diagram

2.4 User Interface Mockups

The following mockups illustrate the proposed user interface (UI) for the RiskDev platform. They are conceptual designs showing how the system presents information and functionality to different types of users. These mockups highlight the interaction flow, the display of package data, vulnerabilities, and project management features, and demonstrate how users can navigate and analyze information within the platform.

2.4.1 Guest (Unregistered User)

This mockup shows the interface available to a Guest. Guests can search for Python packages, view metadata, and explore both direct and inherited vulnerabilities. They can also view detailed vulnerability information, including scores, descriptions, and fixed versions, and have the option to register for an account.

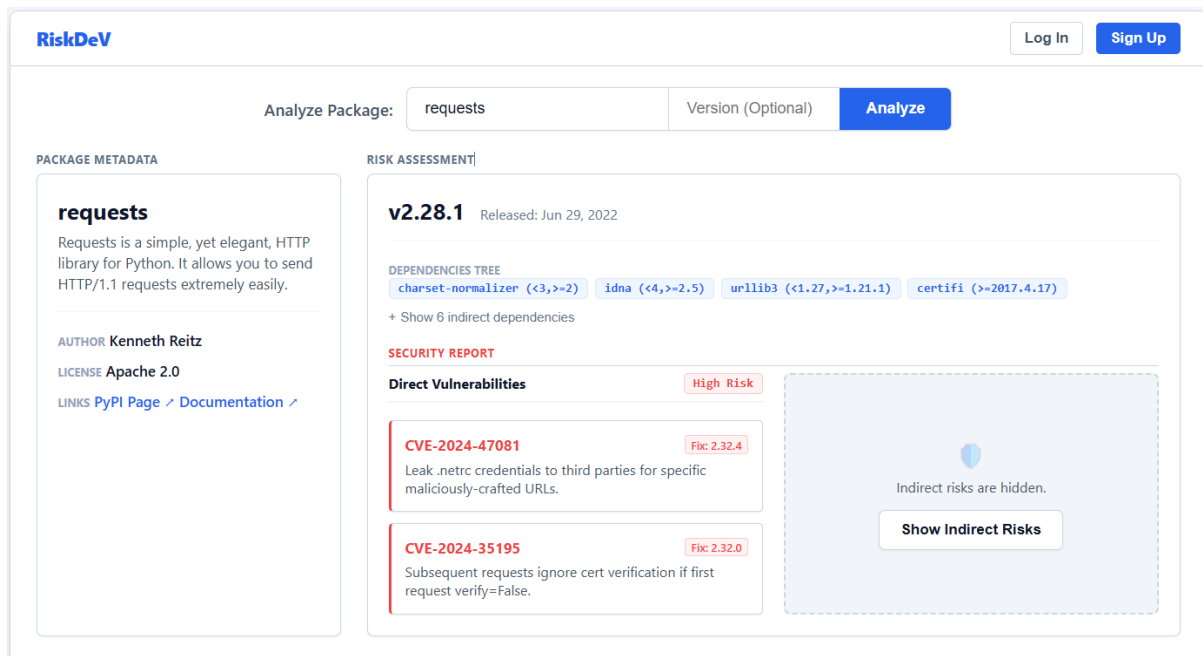


Figure 2.2: Guest User Interface Mockup: The Guest can view package metadata, direct dependencies, and direct vulnerabilities, with buttons to reveal indirect dependencies and inherited vulnerabilities.

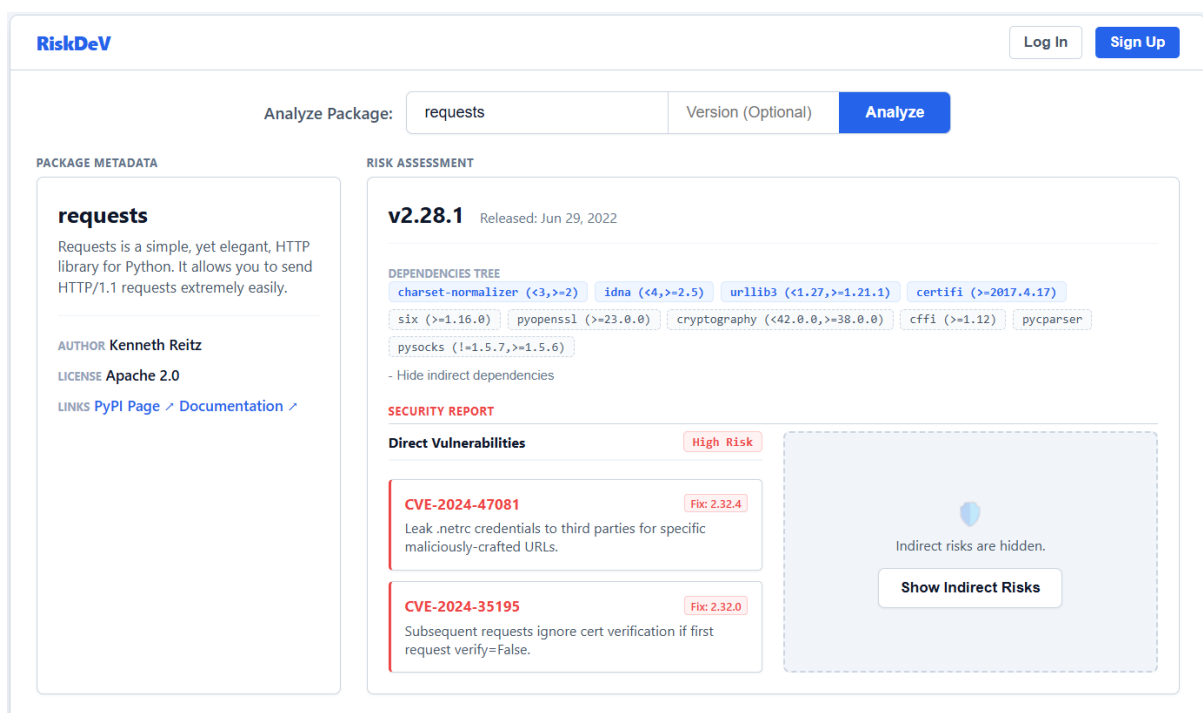


Figure 2.3: Guest User Interface Mockup: The Guest can view detailed package information, including direct dependencies and vulnerabilities, and can display indirect dependencies by clicking a button.

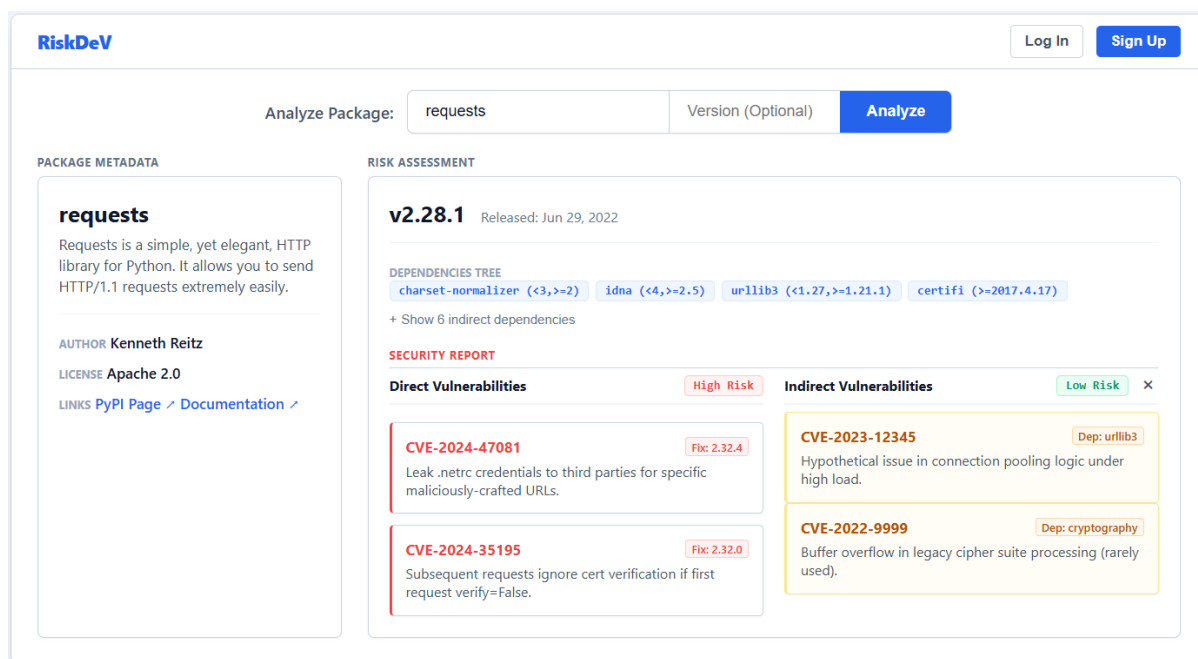


Figure 2.4: Guest User Interface Mockup: The Guest can view detailed package information and inherited vulnerabilities by selecting the appropriate option.

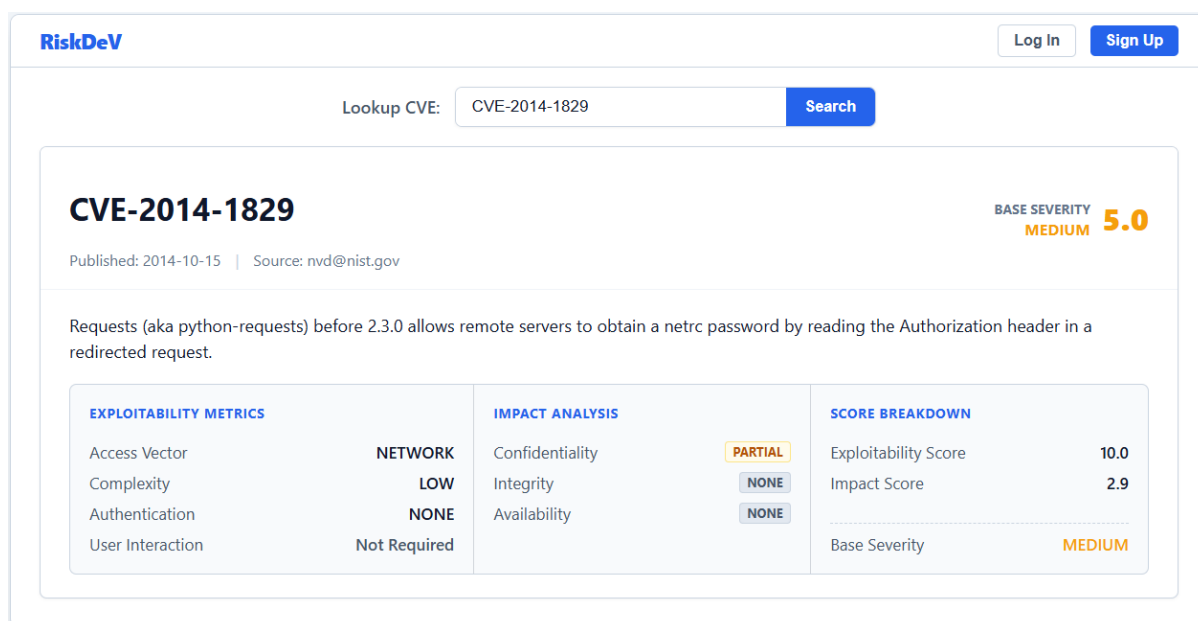


Figure 2.5: Guest User Interface Mockup: The Guest can search for a specific vulnerability using its CVE ID and view all related information, including metrics, description, publication date, and source.

2.4.2 Registered User

This mockup shows the interface available to a Registered User. In addition to all Guest functionalities, Registered Users can create and manage projects, add packages and collaborators, analyze project dependencies and vulnerabilities, and identify safe package versions. The UI also displays aggregated project statistics, including overall risk scores and per-package vulnerability information.

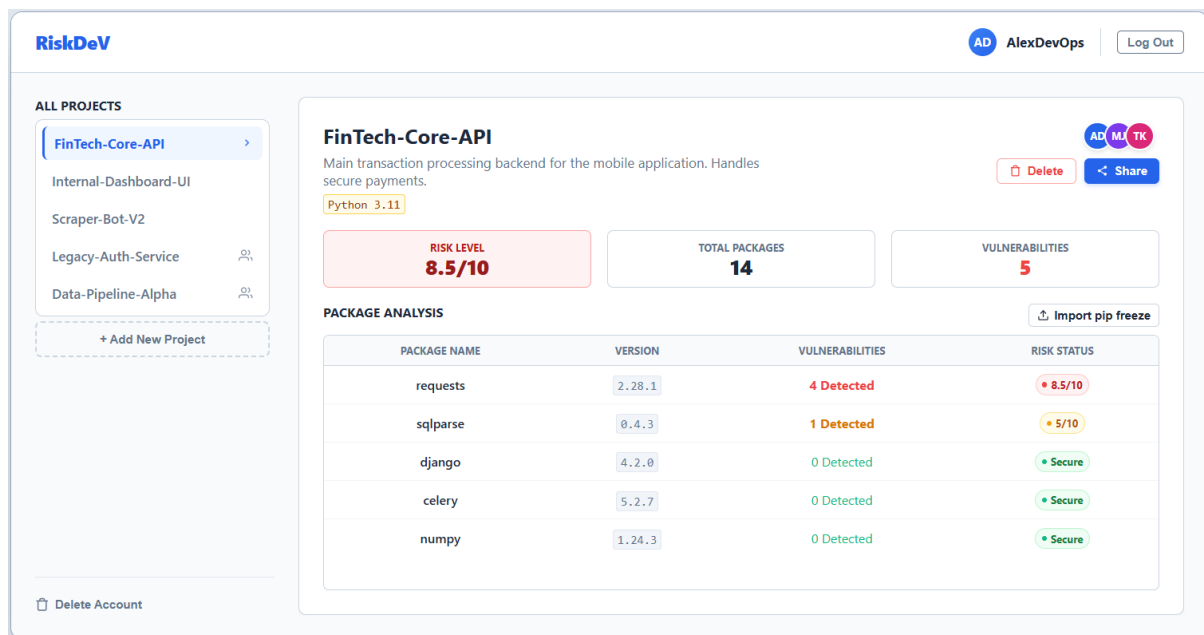


Figure 2.6: Registered User Interface Mockup: The Registered User can view personal projects, including projects they own and projects shared with them. Clicking on a project shows detailed information, such as overall risk level, Python version, collaborators, package list, and per-package risk scores.

2.4.3 Administrator

This mockup shows the interface available to an Administrator. In addition to all Registered User functionalities, Administrators can monitor system usage, track vulnerability trends, analyze packages and dependencies across the ecosystem, and manage user roles.

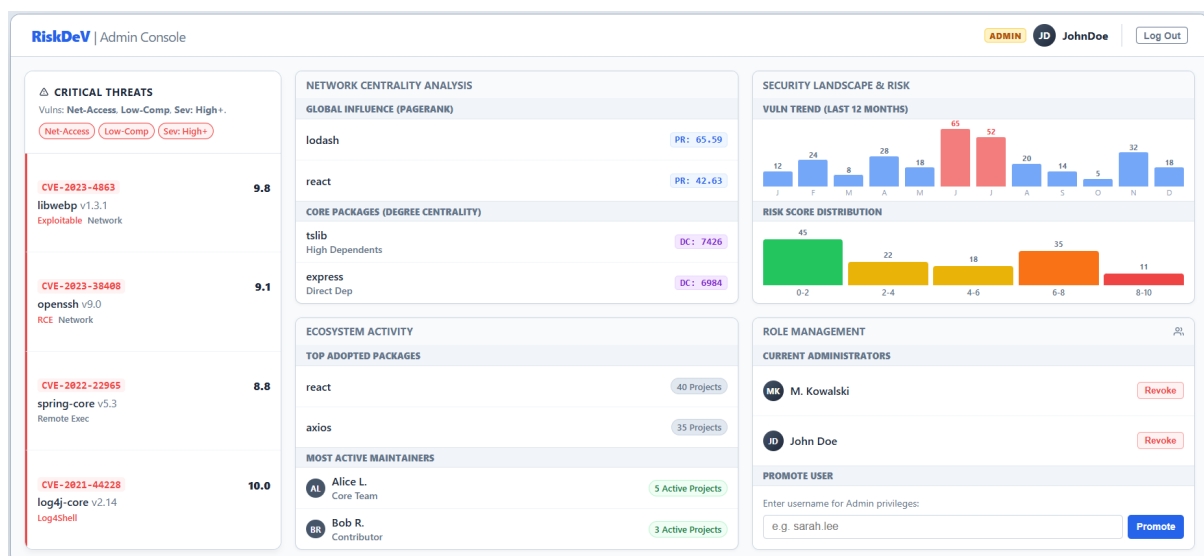


Figure 2.7: Administrator Interface Mockup: The Administrator can view vulnerability trends over time, retrieve system usage statistics including active users and most active projects, analyze packages ranked by PageRank and Degree Centrality, group packages by risk score intervals to assess ecosystem-wide risk, identify the most frequently used dependency packages, examine network-accessible vulnerabilities that are easy to exploit with high or critical severity, and manage user roles, including promoting Registered Users to Administrators or revoking Administrator privileges.

3 Dataset Creation

The dataset for the RiskDev platform was created using a combination of real Python package data from PyPI, vulnerability information from the National Vulnerability Database (NVD) and synthetic data for projects and users. The dataset consists of four JSON files: `vulnerability.json`, `package.json`, `projects.json` and `users.json`.

3.1 Package Metadata and Vulnerabilities

Two Python scripts were used to collect and process package data:

1. `pkg-cve-script.py` fetches metadata and vulnerabilities from PyPI and NVD. It starts from an initial list of packages and retrieves:
 - Package metadata: name, author, description, documentation links, available versions and dependencies.
 - Vulnerabilities for each version from PyPI, if any.
 - Detailed CVE information from NVD, prioritizing CVSS v4.0, then v3.1, v3.0 and v2.0.

This script produces `allInfoPackages.json` (package metadata and dependencies) and `vulnerability.json` (CVE details and metrics).

2. `generate-package-collection.py` processes the raw JSON data (`allInfoPackages.json`) to normalize versions and calculate risk scores:
 - Converts version strings into fixed-length numeric arrays for sorting and comparison.
 - Parses dependencies into structured fields.
 - Calculates a risk score for each package version.

The final output is `package.json`, containing normalized versions, risk scores, structured dependencies and full metadata ready for analysis.

3.2 Synthetic Projects and Users

Two additional scripts were used to generate realistic project and user data:

1. `generate_projects_and_users.py` creates synthetic Python projects and users:
 - Generates projects with unique name, description, Python version and list of packages.
 - Generates users with unique username, email, hashed password and role (registered user or administrator).
 - Uses temporary virtual environments to retrieve realistic package version with `pip list`.

This produces `projects.json` and `users.json`.

2. `score_project_packages.py` enriches projects with risk information:
 - Maps each package version in a project to its risk score and number of vulnerabilities using `package.json`.
 - Updates each project's package entries with `risk_score` and `vulnerabilities_count`.

Workflow

The overall workflow for creating the dataset is as follows:

1. Collect package metadata and CVEs from PyPI and NVD using `pkg-cve-script.py`.
2. Process package versions, normalize them, parse dependencies and calculate risk scores using `generate-package-collection.py`.
3. Generate synthetic projects and users with realistic packages, admin and collaborators using `generate_projects_and_users.py`.
4. Enrich the projects with risk scores and vulnerability counts for each package using `score_project_packages.py`.

This process produces a comprehensive dataset that supports package-level and project-level analysis, including dependency tracking, vulnerability assessment and risk scoring. It enables the RiskDev platform to simulate realistic user projects while leveraging real package and vulnerability data.

3.3 Dataset Size

The table below summarizes the approximate size of each file:

File	Size (MB)
package.json	451.2
vulnerability.json	0.522
projects.json	0.073
users.json	0.032

Table 3.1: Approximate sizes of the JSON files used in the RiskDev dataset.

4 Data Modeling and Structure

The RiskDev platform uses two different database systems, each selected to address a specific set of requirements: MongoDB as the primary data store and Neo4j as the graph database for relationship modeling.

MongoDB was chosen as the primary database due to its flexibility in handling large volumes of semi-structured data. Package metadata, vulnerability details user and project information vary in structure and size, making a document-based model well suited for this type of data. MongoDB allows each package version and vulnerability to be stored as a self-contained document, enabling efficient retrieval of detailed information.

Neo4j was selected to represent and manage the complex relationships between packages, their dependencies and associated vulnerabilities. These elements naturally form a graph structure, where nodes represent packages, versions and vulnerabilities and edges represent dependency links and vulnerability associations. By using a graph database, the system can efficiently perform complex and meaningful queries, such as identifying indirect dependencies, tracing vulnerability propagation across a dependency chain and ranking packages based on their structural importance in the ecosystem and many other types of analysis.

In this architecture, MongoDB serves as the authoritative source for detailed and descriptive data, while Neo4j is used to store and analyze the relational structure of the software ecosystem. This separation of responsibilities allows the system to combine efficient data storage with powerful graph-based analysis.

The following two sections describe in detail the structure of the MongoDB collections and the Neo4j nodes and relationships used in the system.

4.1 MongoDB

MongoDB is used as the primary database of the RiskDev platform and stores all detailed, descriptive and user-generated information. The document-oriented model allows the system to manage data with different structures and sizes while keeping each logical entity self-contained and easy to retrieve.

To support the main platform features, five separate collections were defined, each with a specific responsibility: *package*, *project*, *user*, *vulnerability* and *dlq* (Dead Letter Queue).

4.1.1 Package Collection

The *package* collection stores detailed information about each package version. Each document represents a single version of a Python package and contains metadata, normalized version information, structured dependencies, associated vulnerabilities and a calculated risk score. This structure allows fast access to all relevant data for package analysis and comparison.

```
{
  "_id": {
    "$oid": "697be19c9a7f2025ffd87d3a"
  },
  "package_name": "requests",
  "version": "1.2.0",
  "version_array": [1,2,0,0,0,0],
  "author": "Kenneth Reitz",
  "author_email": "me@kennethreitz.org",
  "description": "...",
  "package_url": "https://pypi.org/project/requests/",
  "documentation": "https://requests.readthedocs.io",
  "upload_time": {
    "$date": "2013-03-31T03:28:47.000Z"
  },
  "requires_dist": [],
  "requires_python": null,
  "vulnerabilities": [
    {
      "cve_id": "CVE-2014-1829",
      "details": "Requests (aka python-requests) before 2.3.0 allows remote servers to obtain a netrc password by reading the Authorization header in a redirected request.",
      "fixed_in": [
        "2.3.0"
      ],
      "link": "https://osv.dev/vulnerability/PYSEC-2014-13"
    },
    {
      "cve_id": "CVE-2018-18074",
      "details": "The Requests package before 2.20.0 for Python sends an HTTP Authorization header to an http URI upon receiving a same-hostname https-to-http redirect, which makes it easier for remote attackers to discover credentials by sniffing the network.",
      "fixed_in": [
        "2.20.0"
      ],
      "link": "https://osv.dev/vulnerability/PYSEC-2018-28"
    }
  ],
  "risk_score": 7.5
}
```

4.1.2 Project Collection

The *project* collection represents user-defined Python projects. Each document contains general project information, the required Python version, the list of installed packages and the users associated with the project. This structure allows Registered Users to analyze their working environment and evaluate the security risk of their dependencies.

```
{
  "_id": {
    "$oid": "697be1de4b7a113127101334"
  },
  "name": "Vertex-Service",
  "description": "Vertex-Service is a Python project using 1 core libraries.",
  "last_update": {
    "$date": "2021-11-24T10:47:57.000Z"
  },
  "python_version": "3.11.14",
  "packages": [
    {
      "name": "numpy",
      "version": "1.26.2"
    },
    {
      "name": "pip",
      "version": "24.0"
    },
    {
      "name": "setuptools",
      "version": "79.0.1"
    }
  ],
  "admin": {
    "username": "michael.ferrari",
    "email": "michael.ferrari@company.io"
  },
  "collaborators": [
    {
      "username": "michael.ferrari",
      "email": "michael.ferrari@company.io"
    },
    {
      "username": "giulia.esposito",
      "email": "giulia.esposito@company.io"
    }
  ]
}
```

4.1.3 User Collection

The *user* collection stores registered user profiles and access control information. Each document includes personal details, a securely hashed password, the assigned role and the list of projects the user is associated with. This collection supports authentication, authorization and project sharing features.


```
{
  "_id": {
    "$oid": "6978e40f347161777763b2ac"
  },
  "username": "francesca.romano",
  "first_name": "Francesca",
  "last_name": "Romano",
  "email": "francesca.romano@example.com",
  "password": "0dc5d26a6a85c6288a3c691c9189c14626e0bf7c08006c0f1482d5964f789f74",
  "role": "ROLE_USER",
  "project_names": [
    "Gamma-Studio"
  ]
}
```

4.1.4 Vulnerability Collection

The *vulnerability* collection stores detailed CVE information retrieved from the NVD. Each document contains the vulnerability description, publication date, CVSS metrics and severity-related attributes. This data is used to enrich both package-level and project-level risk analysis.

```
{
  "_id": {
    "$oid": "697be219778bc3593b501691"
  },
  "published": {
    "$date": "2014-10-15T12:55:05.400Z"
  },
  "description": "Requests (aka python-requests) before 2.3.0 allows remote servers to obtain a netrc password by reading the Authorization header in a redirected request.",
  "metrics": {
    "source": "nvd@nist.gov",
    "type": "Primary",
    "baseSeverity": "MEDIUM",
    "exploitabilityScore": 10,
    "impactScore": 2.9,
    "acInsufInfo": false,
    "obtainAllPrivilege": false,
    "obtainUserPrivilege": false,
    "obtainOtherPrivilege": false,
    "userInteractionRequired": false,
    "version": "2.0",
    "vectorString": "AV:N/AC:L/Au:N/C:P/I:N/A:N",
    "baseScore": 5,
    "accessVector": "NETWORK",
    "accessComplexity": "LOW",
    "authentication": "NONE",
    "confidentialityImpact": "PARTIAL",
    "integrityImpact": "NONE",
    "availabilityImpact": "NONE"
  },
}
```

```
"cve_id": "CVE-2014-1829"
}
```

4.1.5 Dead Letter Queue (DLQ) Collection

The *dlq* collection is used to support asynchronous consistency between MongoDB and Neo4j. All write, update and delete operations are first applied to MongoDB. If the operation succeeds, a confirmation is immediately returned to the user to ensure fast response times. An asynchronous event is then generated to propagate the change to Neo4j.

If the synchronization with Neo4j fails, the event is stored in the DLQ collection. This allows the system to retry the operation later and restore consistency between the two databases without losing updates.

```
{
  "_id": {
    "$oid": "697696306b79d74f4b39a432"
  },
  "eventType": "VersionReleaseEvent",
  "payloadJson": "...",
  "exceptionMessage": "Resolved by Scheduler",
  "failedAt": {
    "$date": "2026-01-25T22:16:16.664Z"
  },
  "retryCount": 0,
  "resolvedAt": {
    "$date": "2026-01-31T22:18:18.102Z"
  },
  "_class": "it.unipi.riskDeV.model.FailedEvent"
}
```

This design allows MongoDB to remain the authoritative and fast-access data source, while Neo4j is kept consistent in the background and used exclusively for relationship-based and graph-oriented analysis.

4.1.6 Document Embedding and Linking

In designing the MongoDB schema for RiskDev, we opted for a **denormalized approach** to optimize read operations.

For the **PackageVersion** collection, instead of separating general package metadata from version-specific information, we embedded the common package details (such as package name, author, description and documentation URL) directly within each version document. While this introduces some redundancy, it aligns perfectly with our access patterns: most queries require both version-specific data (e.g., dependencies, vulnerabilities) and general package metadata simultaneously. Embedding this information eliminates the need for joins or multiple database calls, resulting in more efficient data retrieval.

The **project** collection uses a mixed approach of document embedding and linking. User information (e.g., admin and collaborators) is partially embedded, storing both the username and email. This ensures that queries for project details return the contact information of the project owner or collaborators directly, matching the interface mockup shown in Figure 2.6, where hovering over a collaborator's profile displays both the username and email.

The **user** collection, by contrast, relies entirely on document linking. Each user document contains a list of project names the user is involved in. This allows the system to efficiently check whether a user is a project owner or collaborator for access control purposes. Embedding full project details was unnecessary here, so linking suffices and avoids redundant data storage.

4.2 Neo4j

Neo4j is used in the RiskDev platform to model and manage the complex relationships between packages, versions, projects and vulnerabilities. While MongoDB efficiently handles detailed, document-centric data, Neo4j is optimized for traversing highly connected data, which is crucial for tasks like dependency resolution and impact analysis.

4.2.1 Nodes

The main nodes in the graph are:

Package: Represents an abstract Python library, such as `Flask` or `Requests`.

Version: Represents a specific release of a package, e.g., `Requests 2.27.0`. Version nodes store metadata such as the normalized version array, Python requirements, risk score and documentation link. Versions are central for mapping dependencies and vulnerabilities.

Vulnerability: Represents a known security issue or CVE affecting one or more package versions. Each node stores CVE details, description and a base score.

Project: Represents a user-defined Python project containing a collection of package versions.

4.2.2 Relationships

Relationships capture the interactions between nodes:

- `(:Package)-[:HAS_VERSION]->(:Version)`
Links a package to its specific versions.
- `(:Version)-[:DEPENDS_ON]->(:Version)`
Represents package dependencies, including optional version constraints. This relationship forms the dependency graph used for transitive vulnerability and impact analysis.
- `(:Version)-[:AFFECTED_BY]->(:Vulnerability)`
Connects a version to the vulnerabilities that affect it. This allows for precise risk assessment at the version level.
- `(:Project)-[:USES]->(:Version)`
Connects a project to the specific package versions it uses. Each relationship can store properties about the dependency, enabling further analysis of project risk.

4.2.3 Data Population Process

The graph is populated using a Python-based script (`populate-neo4j-script.py`) that:

- Loads JSON data for packages, vulnerabilities and projects.
- Creates nodes for `Package`, `Version`, `Vulnerability` and `Project` with their properties.
- Resolves dependencies using the version constraints specified in each package's metadata and creates `DEPENDS_ON` relationships only between compatible versions.
- Links versions to vulnerabilities via `AFFECTED_BY`.
- Connects projects to the versions they use via `USES`, creating any missing package and version nodes as needed.

This structure ensures that the graph can efficiently answer complex queries, such as:

- Identifying all direct and transitive dependencies of a package version.
- Finding all versions affected by a particular vulnerability.
- Determining the impact of updating or removing a package within a project.

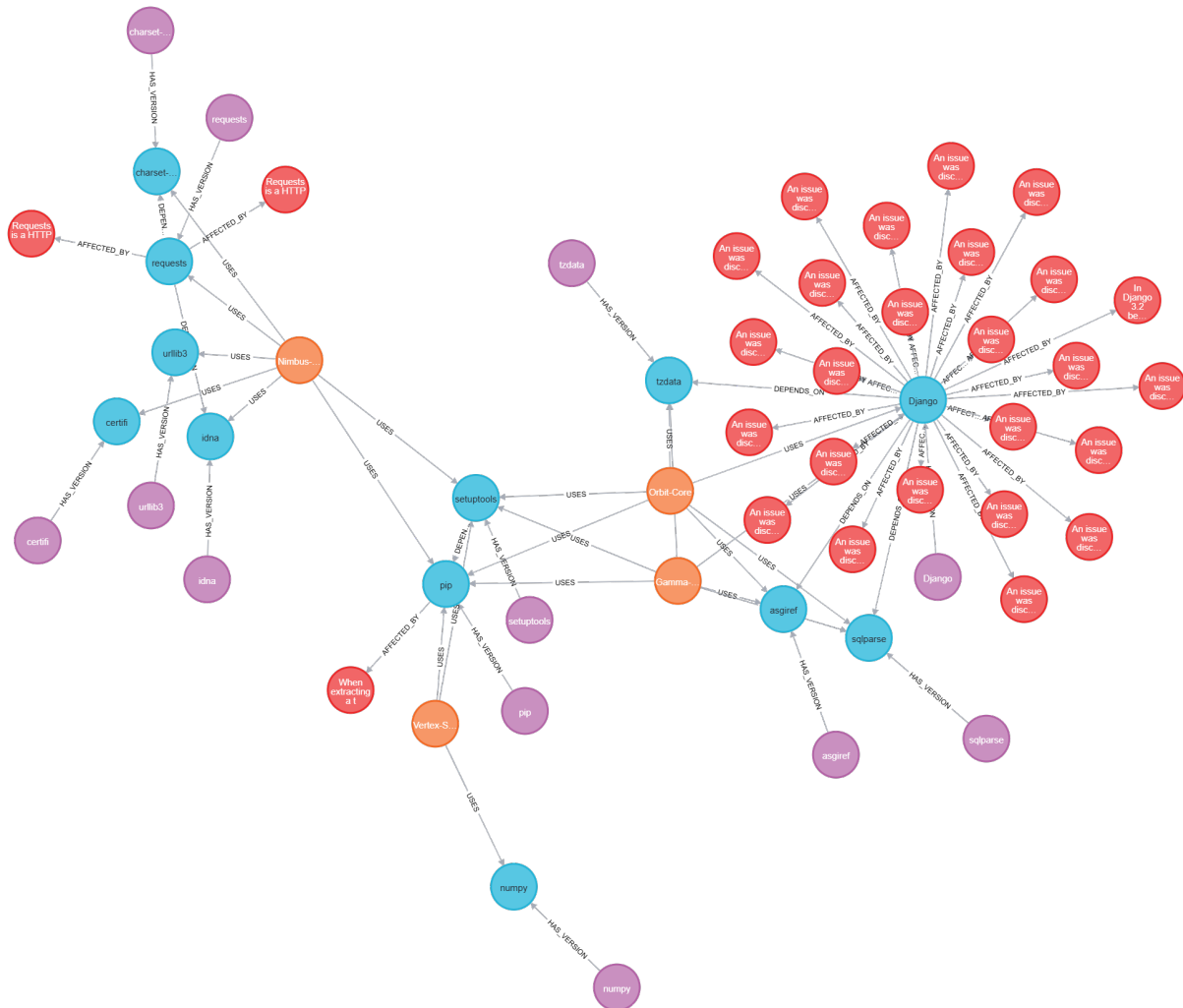


Figure 4.1: Graph visualization representing the complete data model, including Projects (orange), Python Versions (light blue), Packages (purple), and Vulnerabilities (red).

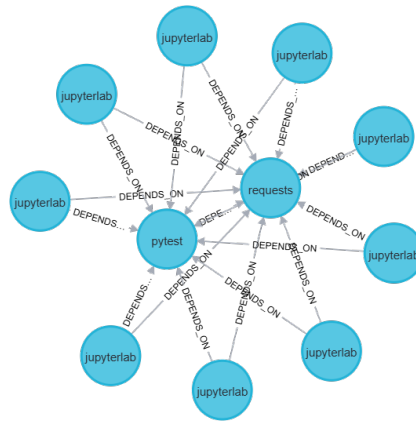


Figure 4.2: Graph visualization filtering only the Version nodes, represented in light blue.

5 Implementation

The backend of the RiskDev platform is built using the **Spring Boot** framework, which provides a robust and flexible foundation for developing scalable Java applications.

The main features and advantages leveraged in this implementation include:

- **RESTful API Development:** Spring Boot simplifies the creation of REST endpoints through annotations such as `@RestController` and `@GetMapping`. This allows for a clean separation between the HTTP interface and business logic, making the code easier to read and maintain.
- **Configuration and Dependency Management:** The Spring container manages the lifecycle and configuration of application components. Additionally, **Lombok** is used to reduce boilerplate code, such as getters, setters and constructors, improving overall readability.
- **Polyglot Persistence Support:** The platform integrates both **MongoDB** and **Neo4j**. Spring Boot provides consistent repository abstractions, enabling seamless interaction with document-oriented and graph databases within the same application context.
- **Security Infrastructure:** **Spring Security** is used to implement authentication and authorization, enforcing Role-Based Access Control (RBAC) for sensitive operations, such as project management and administrative actions. JWT (JSON Web Tokens) support is also integrated for stateless authentication.
- **Validation and OpenAPI Support:** Spring Boot's validation framework ensures input data consistency, while Springdoc OpenAPI provides automatic generation of interactive API documentation.

The project uses Java 21 as the target language and leverages Maven for dependency management and build automation. Key dependencies included in the `pom.xml` are:

- `spring-boot-starter-data-mongodb` and `spring-boot-starter-data-neo4j` for database integration.
- `spring-boot-starter-security` and `jjwt` for authentication and authorization.
- `lombok` to reduce boilerplate code.
- `spring-boot-starter-web` for synchronous web endpoints.
- `springdoc-openapi-starter-webmvc-ui` for API documentation.

By leveraging these frameworks and libraries, the backend implementation focuses on business logic rather than infrastructural details, resulting in a secure, maintainable and scalable architecture.

5.1 Layered Architecture and Design Patterns

To structure the backend code, we adopted a **multi-layered architecture** inspired by the classical MVC (Model–View–Controller) pattern, extended with **Service** and **Repository/DAO** layers and the use of **DTOs (Data Transfer Objects)**. This design improves separation of concerns, maintainability, scalability and testability.

- **Controller Layer:** Receives HTTP requests, maps them to DTOs and delegates business logic to the Service layer. Controllers remain thin and do not contain business logic.

- **Service Layer:** Implements the application's business rules, orchestrates interactions between controllers and repositories and converts between Entities and DTOs.
- **Repository/DAO Layer:** Encapsulates all database access, interacting only with Entities. Spring Data Repositories provide automatic CRUD operations, while custom DAOs can be added for specific behaviors.
- **DTOs:** Transfer data between layers and to external clients, avoiding direct exposure of internal Entities and database structure.
- **Models:** Represent persistent data models that mirror the database schema (MongoDB documents or Neo4j nodes) and contain annotations for persistence and relationships.

The typical request flow in RiskDev follows:

1. Client sends a request (JSON) → converted into DTO
2. Controller receives DTO → calls Service layer
3. Service layer applies business logic → interacts with Repository/DAO
4. Repository returns Entity → Service converts Entity to DTO
5. Controller returns DTO as JSON response

This architecture ensures a clean separation between internal data models and external representation, supports multiple data sources and allows future extensions, such as integrating new databases or modifying business logic, without affecting other layers.

5.2 Logging Configuration

RiskDev implements a robust logging system using **Logback**, which is integrated with Spring Boot by default. Logging serves multiple purposes, including debugging, monitoring and auditing system operations.

The key aspects of the logging setup are:

- **File-based Logging:** All application logs are written to a rolling log file located at `logs/app.log`.
- **Daily Rotation and Retention:** Logs are rotated daily, with up to 30 days of history preserved. Old log files are cleaned automatically on application startup to avoid excessive disk usage.
- **Custom Log Pattern:** Each log entry includes the timestamp, thread, log level, logger name, and message, providing clear and consistent traceability for events.
- **Console Output Disabled:** Console logging is disabled in production to reduce noise and improve performance, focusing all logging to the file system.

This configuration ensures that RiskDev maintains a persistent and organized log history, which is crucial for debugging, auditing user actions, and monitoring the behavior of the backend system over time.

5.3 Project Structure and Package Organization

5.3.1 `it.unipi.riskDeV.config`

The `it.unipi.riskDeV.config` package contains the core configuration classes for the RiskDeV Spring Boot application. These classes manage cross-cutting concerns such as security, API documentation, asynchronous processing, JSON serialization, and database transactions. Below is an overview of the main components:

- **SecurityConfig.java:** Configures the security layer of the application using Spring Security. Key responsibilities include:
 - **Authentication and Authorization:** Integrates a custom `JwtAuthenticationFilter` before the standard authentication filter to handle JWT-based authentication. Role-Based Access Control (RBAC) is enforced as follows:
 - * Public access is allowed for authentication endpoints (`/api/auth/**`) and Swagger documentation.
 - * Read-only access (GET) to vulnerabilities and packages is publicly available, while modification endpoints require `ROLE_ADMIN`.
 - * Project and user management endpoints are restricted to authenticated users with `ROLE_USER` or `ROLE_ADMIN`.
 - **Session Management:** Configured to `STATELESS`, ensuring that every request is independently authenticated with JWT tokens, in line with REST principles.
 - **Password Encoding:** Defines a `BCryptPasswordEncoder` bean for secure password hashing.
- **OpenApiConfig.java:** Configures the OpenAPI 3.0 specification (Swagger) for the application. It sets the API metadata (title, version, description) and defines the `bearerAuth` security scheme, allowing JWT-based authorization directly in the Swagger UI.
- **AsyncConfig.java:** Enables asynchronous method execution and task scheduling using Spring's `@EnableAsync` and `@EnableScheduling` annotations. This allows background processing and periodic tasks to run without blocking main application threads.
- **JacksonConfig.java:** Customizes the application's `ObjectMapper` for JSON serialization and deserialization. Main features:
 - Registers `JavaTimeModule` and `ParameterNamesModule` to correctly handle Java 8 date/time types and constructor parameter names.
 - Disables `WRITE_DATES_AS_TIMESTAMPS` to produce ISO-8601 formatted dates.
 - Configures deserialization to ignore unknown properties and accept empty strings as null objects.

5.3.2 `it.unipi.riskDeV.controller`

The `it.unipi.riskDeV.controller` package defines the RESTful API of the RiskDeV application and serves as the main entry point for client interactions. Controllers are responsible for HTTP request mapping, input validation, response serialization, and delegation of business logic to the corresponding service layer. All endpoints are annotated with OpenAPI (Swagger) metadata to enable automatic API documentation and support JWT-based authentication.

Controllers follow a thin-layer pattern: they interact with DTOs for requests and responses, delegate core business logic to services, and leverage centralized exception handling and response mapping to provide consistent HTTP responses.

The main controllers are:

- **AdminController:** Exposes endpoints for administrative operations and system-wide analytics, accessible only to users with `ROLE_ADMIN`. Responsibilities include:
 - Retrieving ecosystem metrics, such as package centrality (Degree Centrality, PageRank) and top contributors.
 - Performing risk aggregation and detecting critical "Perfect Storm" vulnerabilities.
 - Managing user roles, including promotion and demotion of administrators.
- **AuthController:** Serves as the public access point for authentication and user registration. Responsibilities include:
 - Registering new users and issuing JWT tokens upon successful registration.
 - Authenticating existing users and returning JWT tokens for stateless access to protected API endpoints.
 - Enforcing input validation and returning structured error responses for invalid data or duplicate accounts.
- **PackageController:** Handles operations on Python packages and their versions, supporting both public queries and authenticated updates. Responsibilities include:
 - Retrieving package metadata and version-specific information.
 - Listing direct and reverse dependencies for a given package version.
 - Identifying safe versions with no known vulnerabilities, including transitive vulnerability checks.
 - Publishing new package versions and updating existing metadata or version details (dependencies, vulnerabilities, Python requirements).
 - Deleting specific package versions from both the database and the dependency graph.
- **ProjectController:** Provides endpoints for authenticated users to create and manage projects as collaborative workspaces. Responsibilities include:
 - Creating, updating, and deleting projects.
 - Adding, updating, and removing packages in a project.
 - Managing collaborators, including adding, listing, and removing project collaborators.
 - Enforcing access control to ensure that only owners can manage collaborators and sensitive project operations.
 - Monitoring project dependencies and triggering automated risk recomputation upon changes.
- **UserController:** Manages self-service user operations, including:
 - Retrieving and updating the profile information of the currently authenticated user.
 - Listing projects in which the user participates, either as owner or collaborator.
 - Permanent account deletion with automatic cleanup of user-related references in projects.
 - Enforcing uniqueness of usernames and validation of updated profile fields.
- **VulnerabilityController:** Provides security-focused functionality, including:

- Retrieving detailed CVE information, including description, publication date, and security metrics.
- Listing packages and versions affected by a specific vulnerability.
- Performing transitive vulnerability analysis over a package's dependency graph.
- Creating, updating, and deleting vulnerability records in the system.
- Ensuring controlled access to sensitive operations via JWT-based authentication.

5.3.3 `it.unipi.riskDeV.service`

The `it.unipi.riskDeV.service` package contains the core business logic of the RiskDeV application. Services act as an intermediary between controllers and the persistence layer, ensuring transactional consistency, enforcing domain validation rules, and coordinating complex workflows. They also publish domain events via the `ApplicationEventPublisher` to notify other components of significant state changes.

Result Wrapper: All service methods return a `Result<T>` object, representing either a successful computation or a failure. This wrapper enforces explicit handling of errors and avoids unchecked exceptions in business logic. It has two main variants:

- **Result.Success<T>:** Encapsulates a successful result and holds the returned data of type `T`. For example, fetching a package version successfully returns `Result.Success<PackageVersionDTO>`.
- **Result.Failure<T>:** Encapsulates a domain error that occurred during execution, represented by `DomainError` (e.g., `NotFound`, `AlreadyExists`, `InvalidOperation`). This ensures that controllers can consistently map failures to structured HTTP responses.

ApplicationEventPublisher: In RiskDeV, services use `ApplicationEventPublisher` to trigger asynchronous or cross-cutting workflows through domain events. This allows the system to separate main data writes from secondary updates, improving performance and consistency.

Specifically, the main writes happen synchronously in MongoDB, ensuring fast and reliable data storage. After that, an event is published to handle any necessary updates in Neo4j, keeping the dependency graph consistent with the newly saved data.

This mechanism also allows calculations of derived or aggregated fields to be done asynchronously, so they do not slow down API responses. Complex updates or calculations are performed in the background, ensuring a fast response for clients while maintaining consistency across the system.

This event-driven pattern ensures decoupling between services and promotes eventual consistency across multiple storage systems (relational, document, and graph databases).

Main Services:

- **AdminService:** Implements administrative features, including role management and computation of global analytics such as package centrality metrics, risk distributions, and detection of critical "Perfect Storm" vulnerabilities.
- **AuthService:** Manages authentication workflows, ensuring unique credentials during registration, securely hashing passwords with BCrypt, and generating JWT tokens for stateless authentication.
- **PackageService:** Handles package and version management, ensuring consistency between the document database and the dependency graph.

- **ProjectService:** Coordinates project lifecycle management and access control, differentiating between owners and collaborators. Modifying project dependencies triggers automatic risk recomputation.
- **UserService:** Manages user profiles and accounts, enforcing uniqueness constraints and proper validation.
- **VulnerabilityService:** Maintains CVE data and performs dependency graph traversal to identify transitive vulnerabilities.

5.3.4 `it.unipi.riskDeV.repository`

This package constitutes the persistence layer of the application, implementing the "Polyglot Persistence" pattern. It abstracts the data access logic for two distinct database technologies: MongoDB (for document storage) and Neo4j (for graph-based dependency analysis).

`it.unipi.riskDeV.repository.documentDB`

This package contains interfaces extending `MongoRepository` to interact with the MongoDB database. These repositories primarily handle CRUD operations for the system's core entities and support specific query methods for filtering and retrying operations.

- **UserRepository, ProjectRepository, VulnerabilityRepository:** These interfaces provide standard data access methods for users, projects and vulnerabilities. They include custom finders to enforce uniqueness constraints.
- **PackageVersionRepository:** This repository manages the storage of package metadata. It provides methods to retrieve the latest version of a package by sorting its version array and to filter packages based on their risk scores, allowing identification of safe versions. The repository also supports atomic partial updates on risk scores using the `@Update` annotation, which avoids the need to fetch and save the entire document.
- **FailedEventRepository:** This repository supports the application's fault tolerance mechanism by managing the Dead Letter Queue (DLQ) collection. It provides a method to fetch unresolved events with a retry count below a specific threshold (`findByResolvedAtIsNullAndRetryCountLessThan`), serving as the backbone for the asynchronous retry service.

`it.unipi.riskDeV.repository.graphDB`

This package contains interfaces extending `Neo4jRepository`. These components leverage Cypher queries and the Neo4j Graph Data Science (GDS) library to perform complex topology analysis on the dependency graph.

- **PackageGraphRepository:** This repository handles global graph metrics and centrality analysis. It can calculate Degree Centrality to measure package "popularity" based on incoming dependencies and use the Neo4j GDS PageRank algorithm to assess the overall importance of packages within the entire dependency graph.
- **PackageVersionGraphRepository:** This is the core repository for dependency traversal. It handles the creation of graph relationships (`DEPENDS_ON`, `AFFECTED_BY`) and executes deep recursive queries. Notably, it implements `findRecursiveVulnerabilities`, which uses variable-length path matching (`[[:DEPENDS_ON*]]`) to trace transitive vulnerabilities deep in the dependency tree.

- **ProjectGraphRepository**: This repository bridges the user domain with the package graph. It manages the **USES** relationship, linking **ProjectNode** entities to specific **PackageVersionNode** entities, allowing the system to map a user's project dependencies onto the global graph.
- **VulnerabilityGraphRepository**: This repository facilitates reverse-lookup operations. It allows the system to identify all package versions directly affected by a specific CVE ID by traversing the **AFFECTED_BY** relationship in reverse.

5.3.5 it.unipi.riskDeV.model

it.unipi.riskDeV.model.documentDB

This package defines the data model for the MongoDB document store. It includes the main entities of the domain and their embedded sub-documents.

- **User**: Represents a registered user. It stores authentication details (hashed password), profile information, the assigned role and a list of references to the projects the user is involved in.
- **Project**: It contains the project's metadata, a list of **Collaborator** embedded objects (denormalized for read performance) and a list of **ProjectPackage** objects. The latter is a snapshot of the installed packages, including their calculated risk score at the time of the last update.
- **PackageVersion**: Represents a specific version of a software library and serves as a central aggregate in the system. It contains general metadata such as author, license, and upload time, along with runtime dependency information through **Constraints** and embedded security data via **EmbeddedVulnerability** objects for immediate access to known vulnerabilities.
- **Vulnerability**: Stores detailed information about Common Vulnerabilities and Exposures (CVEs). It utilizes a **Map<String, Object>** structure to handle the schema variability of CVSS metrics. The **vulnerabilityMetrics** sub-package manages the polymorphic deserialization of these metrics into specific classes (**Metric20**, **Metric30**, **Metric40**) depending on the CVSS version.
- **FailedEvent**: Implements the Dead Letter Queue (DLQ) pattern. It stores asynchronous events that failed to process, capturing the exception message, payload and retry count to ensure system reliability and eventual consistency.

it.unipi.riskDeV.model.graphDB

This package defines the entities mapped to the Neo4j Graph Database. These classes represent the nodes in the property graph and are designed to facilitate efficient traversal algorithms.

- **PackageNode**: Represents the abstract concept of a software library, acting as an aggregation root. It connects to all its specific versions via the **HAS_VERSION** relationship, allowing queries to group analytics by package identity.
- **PackageVersionNode**: This is the central node of the dependency graph. It represents a specific release of a package.
- **ProjectNode**: Represents a user's project within the graph. It serves as the entry point for personalized analysis, linking to the **PackageVersionNodes** used by the project via the **USES** relationship.
- **VulnerabilityNode**: Represents a known security flaw (CVE). It stores the **baseScore** to allow severity-based filtering during impact analysis queries. These nodes are linked to affected software versions via the **AFFECTED_BY** relationship.

5.3.6 `it.unipi.riskDeV.DAO`

The `it.unipi.riskDeV.DAO` package contains Data Access Objects (DAOs) that implement complex, custom queries on MongoDB using the `MongoTemplate`. Unlike standard repositories, DAOs provide fine-grained control over aggregation pipelines, sorting, filtering, and data transformation, enabling analytics and reporting features that go beyond simple CRUD operations.

- **PackageDAO:** This DAO performs aggregation operations on package data. For example, it can compute risk buckets by grouping packages based on their risk scores, counting the number of packages in each interval, and producing a summary that includes the total number of packages and the distribution across risk categories. This allows administrative dashboards to visualize the overall risk landscape in the ecosystem.
- **ProjectDAO:** This DAO provides methods to analyze project usage and contributions. It can retrieve the most used packages overall or within the last month, counting the frequency of specific package versions across all projects. It also identifies top collaborators by aggregating the number of contributions per user over a given time window. The DAO leverages MongoDB's aggregation framework to perform these calculations efficiently within the database, reducing application-side processing.
- **VulnerabilityDAO:** This DAO supports security-focused analytics. It can compute trends of vulnerabilities over the last year by aggregating publication dates by month, enabling visualization of how vulnerabilities evolve over time. It also implements the "Perfect Storm" vulnerability query, selecting highly critical vulnerabilities with specific metrics (e.g., network access, no authentication, low complexity, high or critical severity), sorted by base score and limited to the top N results. These operations allow administrators to quickly identify the most impactful vulnerabilities in the ecosystem.

5.3.7 `it.unipi.riskDeV.DTO`

The `it.unipi.riskDeV.DTO` package contains Data Transfer Objects (DTOs) used throughout the RiskDeV backend to structure input and output data between controllers, services, and external clients. DTOs enforce validation, standardize API responses, and decouple internal entity models from external representations. The package is organized into sub-packages for specific domains: `admin`, `packageVersion`, `project`, `user`, and `vulnerability`.

- **Admin DTOs:** These DTOs are specifically designed to model the results of administrative queries and analytics in the RiskDeV system. These DTOs are, for example, returned by endpoints in the `AdminController`, providing structured data for various ecosystem-wide analyses, such as package centrality, risk aggregation, contributor activity, and vulnerability trends.
- **PackageVersion DTOs:** These DTOs model Python packages and their versions. For example, `AddPackageVersionDTO` captures metadata such as package name, version, author, description, documentation URLs, Python requirements, dependency strings, and embedded vulnerability data. Validation annotations ensure required fields are present, email formats are correct, and string lengths conform to expected constraints. This allows safe and consistent ingestion of new package versions into the system.
- **Project DTOs:** Project-related DTOs, such as `ProjectCreationDTO`, encapsulate all necessary information for creating or updating projects. They include the project name, description, target Python version, and a list of installed packages. Validation ensures that required fields are not null and that collections like the package list are properly structured and validated.

User DTOs: User DTOs handle authentication and profile management. `AuthResponseDTO` provides JWT tokens and basic user information upon successful login, while `UpdateProfileDTO` allows clients to safely update usernames, emails, passwords, and personal details with validation rules to ensure uniqueness, proper formats, and secure password policies.

Vulnerability DTOs: Vulnerability DTOs model CVE data and associated metrics.

`VulnerabilityDTO` converts MongoDB `Vulnerability` entities into structured output, including CVE identifiers, publication dates, descriptions, and CVSS metrics. Polymorphic deserialization through `BaseMetricDTO` and its subclasses (`Metric20DTO`, `Metric30DTO`, `Metric40DTO`) allows flexible handling of different CVSS versions while preserving the base score for risk calculations. Annotations also support Swagger/OpenAPI documentation for interactive API exploration.

5.3.8 `it.unipi.riskDeV.security`

The `it.unipi.riskDeV.security` package implements the authentication and authorization mechanisms of RiskDeV using JWT (JSON Web Tokens) and Spring Security. Its main components are designed to ensure stateless, secure access control to the platform's REST endpoints.

- **`JwtAuthenticationFilter`:** This filter extends Spring Security's `OncePerRequestFilter` and is executed for every incoming HTTP request. It extracts the JWT token from the `Authorization` header and validates it. If the token is valid, the filter retrieves the username and role from the token, creates an `Authentication` object, and stores it in the `SecurityContextHolder`. This enables Spring Security to enforce role-based access control for downstream services and controllers. Invalid or missing tokens are logged and skipped, allowing public endpoints to be accessed without authentication.
- **`JwtUtil`:** This utility class handles JWT token creation and validation. It generates tokens containing user-specific claims such as the username and role, with a configurable expiration time. The token is signed using a secret key, ensuring integrity and authenticity. `JwtUtil` also provides methods to extract claims (username and role) from a token and to validate the token's signature and expiration. By separating token management from the authentication filter, the class centralizes JWT handling and simplifies token-related operations.

5.3.9 `it.unipi.riskDeV.results`

The `it.unipi.riskDeV.results` package provides a structured approach for handling success and failure outcomes in the RiskDeV backend. It introduces two key concepts: `Result<T>` and `DomainError`, together with a utility for mapping these results to HTTP responses.

- **`Result<T>`:** The `Result` interface is a generic wrapper representing either a successful computation or a failure. It has two implementations: `Success<T>` and `Failure<T>`. `Success<T>` encapsulates a successful operation and holds the resulting data of type `T`, while `Failure<T>` encapsulates a `DomainError` describing what went wrong. This design enforces explicit handling of errors in the service layer and prevents unhandled exceptions from propagating to controllers. Convenience static methods `Result.success(data)` and `Result.failure(error)` simplify the creation of `Result` instances.
- **`DomainError`:** This sealed interface defines all possible domain-level errors in the application. Its implementations include:

- `NotFound` for missing resources.
 - `AlreadyExists` for conflicts when creating duplicates.
 - `SystemError` for unexpected runtime exceptions.
 - `InvalidCredentials` for failed authentication attempts.
 - `ValidationFailed` for invalid input or business rule violations.
 - `AccessDenied` for forbidden operations.
 - `InvalidOperation` for attempts that violate domain logic.
- **RestResponseMapper:** This component converts `Result<T>` objects into HTTP responses. Successful results are returned with the specified HTTP status (e.g., 200 OK, 201 Created), while failures are mapped to the appropriate status code based on the type of `DomainError`. For example, `NotFound` results in 404, `AlreadyExists` returns 409, and `SystemError` triggers a 500 Internal Server Error. This mapper ensures consistent error handling across the API, standardizes response payloads through `ErrorResponseDTO`, and logs critical system errors for monitoring and debugging purposes.

5.3.10 `it.unipi.riskDeV.util`

The `it.unipi.riskDeV.util` package provides utility classes that support various backend operations in RiskDeV, primarily around dependency parsing, version normalization, database updates, and safe execution of service logic. These helpers abstract repetitive or complex tasks, ensuring consistency and reducing boilerplate across services.

- **DependencyParser:** This class is responsible for parsing Python package dependency strings into structured `Constraints` objects. It uses regular expressions to extract the package name and version constraints such as `>=`, `<=`, `>`, `<`, `==`, and `!=`. It ensures that all version constraints are captured and correctly stored for later evaluation during dependency resolution.
- **Helper:** The `Helper` component provides database-oriented utility methods for managing package and project data in MongoDB. Its main functions include:
 - Resolving dependencies against the current package graph, using version constraints to identify applicable package versions.
 - Updating reverse dependencies when a new version of a package is released or modified.
 - Updating general package metadata in bulk, while ensuring that only the provided fields are modified.
 - Computing the maximum base CVSS score among a list of CVE identifiers through MongoDB aggregation, supporting risk calculations across dependencies.

These methods leverage `MongoTemplate` for flexible queries and updates, including element matching, range filtering, and aggregation pipelines.

- **VersionParser:** This utility converts semantic version strings into normalized integer arrays to facilitate comparisons. Special version labels such as `alpha`, `beta`, `rc`, `post`, and `pl` are mapped to numerical weights to allow consistent ordering of pre-release and post-release versions. Each version array is padded to a fixed length to simplify sorting and comparison operations.
- **ResultExecutor:** This helper wraps potentially unsafe operations in a safe execution context, returning a `Result<T>` that captures either success or failure. Any unhandled exceptions are caught, logged, and converted into a `DomainError.SystemError`, ensuring that service methods always return a consistent `Result` object without propagating runtime exceptions.

5.3.11 it.unipi.riskDeV.async

The **async** package implements the event-driven synchronization mechanism between MongoDB and Neo4j, alongside the Dead Letter Queue infrastructure for handling transient failures in asynchronous operations.

- **Events:** The event domain is modeled using sealed interfaces that restrict implementation to a known set of permitted subtypes, enabling exhaustive pattern matching. Four distinct event, each specified by many records, are defined:
 - **PackageEvent:**
 - * **VersionRelease:** Triggers propagation of the new package version to the graph database.
 - * **UpdateDocumentation:** Signals modification of the documentation URL for a package, triggering propagation of the new URL to all associated version nodes in the graph database.
 - * **UpdatePackageVersion:** Indicates structural changes to a specific version's dependency constraints or vulnerability associations, requiring replacement operations in the graph.
 - * **DeletePackageVersion:** Commands the removal of a specific version node and all its incident relationships from the graph.
 - **ProjectEvent:**
 - * **ProjectCreated:** Initializes a new project structure in the graph database with its initial dependency set and establishes admin ownership.
 - * **ProjectDeleted:** Triggers cascade deletion of the project node and orchestrates cleanup of collaborator associations.
 - * **ProjectPackagesUpdated:** Signals a complete replacement of the project's dependency tree, requiring a synchronization of dependency with the new package set.
 - * **CollaboratorAdded:** Establishes membership relationships between a user and a project.
 - * **CollaboratorRemoved:** Dissolves user-project associations and ensures cleanup of reverse references to maintain a correct information denormalization.
 - * **CalculateRiskMetrics:** Updates project metadata.
 - **VulnerabilityEvent:**
 - * **VulnerabilityCreated:** Introduces a new CVE node into the graph with base severity score and textual description, making it available for attachment to affected packages.
 - * **VulnerabilityUpdated:** Modifies the severity metrics or description of an existing vulnerability node to reflect updated threat information.
 - * **VulnerabilityDeleted:** Removes the vulnerability node and breaks all its **AFFECTED_BY** relationships with package versions.
 - **UserEvent:**
 - * **UserUpdated:** Propagates identity changes across all project nodes where the user appears as collaborator to maintain denormalized consistency.

To enable polymorphic deserialization from the DLQ where failed events are stored as JSON strings, each sealed interface employs Jackson type info metadata that embeds a discriminatory property in the serialized payload, allowing the system to reconstruct the correct record type during retry operations without maintaining external type mappings.

- **Listeners:** Event consumption is performed by **GraphSyncListener** and **ProjectListener**, both executed within **@Async** managed threads to prevent blocking of HTTP request threads. The

GraphSyncListener handles three categories of event by projecting state changes into Neo4j. It uses defensive try-catch blocks where successful processing completes silently, while any exception triggers delegation to **FailedEventService** to persist the original event and error context in the DLQ, allowing the HTTP transaction to complete successfully while the error is queued for retry. The **ProjectListener** manages user and project events, processing collaborator additions and removals through the user service, handling project deletion cleanup, and evaluating risk metrics.

- **Handlers:** The Strategy pattern enables interchangeable strategies for DLQ event recovery. The **EventHandler** interface declares a capability predicate (**canHandle**) identifying supported event types, and an execution method (**handle**) accepting the JSON payload. The scheduler filters its injected handler collection to select the appropriate strategy for each failed event.

Four strategies implement type-specific recovery:

- **PackageEventHandler:** Processes version lifecycle events, coordinating risk score persistence in MongoDB via **DocumentService** and Neo4j mutations via **GraphService**;
- **ProjectEventHandler:** Manages project structural changes and collaborator updates;
- **VulnerabilityEventHandler:** Handles CVE node creation, modification, and deletion in the graph database;
- **UserEventHandler:** Propagates identity attribute changes across project collaborator.

Each strategy deserializes the payload into the corresponding sealed interface and uses exhaustive switch pattern matching to route execution to the correct record handler. This two-level dispatch (strategy selection by event type, then subtype matching) enforces compile-time exhaustiveness and reduces the risk of missed cases when new variants are introduced.

- **Scheduler:** The **DlqRetryScheduler** processes failed events in batches. Every five minutes it fetches a limited, paginated set of unresolved events (no **resolvedAt** and fewer than five retries). For each event, it selects the first **EventHandler** whose predicate matches the stored type and executes it. On success, it stores the resolution timestamp and outcome; on failure, it increments the retry counter and updates the last-attempt timestamp. Events that exceed the retry threshold are left in the DLQ but excluded from polling, requiring manual intervention.
- **FailedEventService:** This component persists failed events to the Dead Letter Queue. It stores the event type (simple class name), serializes the payload to JSON, records the timestamp, and initializes the retry counter.
- **GraphService:** Maps domain events to Neo4j graph mutations, establishing **DEPENDS_ON** relationships for projects, **HAS_VERSION** relationships for packages, and **AFFECTED_BY** relationships for vulnerabilities. Resolves dependency constraints to entity nodes, performs edge replacement for updates and propagates documentation changes.
- **DocumentService:** Provides targeted MongoDB updates needed to keep document data consistent with new package version releases.

6 Relevant Queries

This chapter presents the set of APIs exposed by the system, covering both MongoDB- and Neo4j-based operations. It includes examples of standard CRUD operations as well as aggregation and analytical queries used to retrieve and process structured and graph-based data.

For detailed API documentation, including request and response formats, parameter descriptions and HTTP status codes (e.g., 200, 400, 404, 500), please refer to the following link: <https://filippogambelli.github.io/RiskDev/>.

6.1 CRUD Operations

6.1.1 Vulnerability Controller

GET /api/vulnerabilities/{CVEID}

Retrieves detailed information about a specific vulnerability, such as the publication date, description and security metrics.

```
1 public Result<VulnerabilityDTO> getVulnerabilityByCveID(String cveId) {
2
3     var optionalVulnerability = vulnerabilityRepository.findByCveId(cveId);
4     if (optionalVulnerability.isEmpty()) {
5         return new Result.Failure<>(new DomainError.NotFound("Vulnerability not found."));
6     }
7
8     return new Result.Success<>(new VulnerabilityDTO(optionalVulnerability.get()));
9 }
```

Figure 6.1: Service implementation of getVulnerabilityByCveID

PUT /api/vulnerabilities/{CVEID}

Updates the information of an existing vulnerability identified by its CVE ID.

```
1 public Result<MessageResponseDTO> updateVulnerability(String cveId, UpdateVulnerabilityDTO dto) {
2     if (!cveId.equals(dto.getCveId())) {
3         return new Result.Failure<>(new DomainError.InvalidCredentials("The CVE ID in the URL does not match the request body."));
4     }
5
6     var vulnerabilityOpt = vulnerabilityRepository.findByCveId(cveId);
7     if (vulnerabilityOpt.isEmpty()) {
8         return new Result.Failure<>(new DomainError.NotFound("Vulnerability does not exist, impossible to update."));
9     }
10
11     Vulnerability vulnerability = new Vulnerability(dto, vulnerabilityOpt.get());
12     vulnerabilityRepository.save(vulnerability);
13
14     eventPublisher.publishEvent(new VulnerabilityEvent.VulnerabilityUpdated(vulnerability.getCveId(), vulnerability.getDescription(), vulnerability.getBaseScore()));
15     return new Result.Success<>(new MessageResponseDTO("Vulnerability updated successfully"));
16 }
17 }
```

Figure 6.2: Service implementation of updateVulnerability

DELETE /api/vulnerabilities/{CVEID}

Deletes the vulnerability identified by the specified CVE ID from the system.

```

1 public Result<MessageResponseDTO> deleteVulnerability(String cveId) {
2     var vulnerabilityOpt = vulnerabilityRepository.findByCveId(cveId);
3
4     if (vulnerabilityOpt.isEmpty()) {
5         return new Result.Failure<>(new DomainError.NotFound("Vulnerability not found."));
6     }
7
8     Vulnerability vulnerability = vulnerabilityOpt.get();
9     vulnerabilityRepository.delete(vulnerability);
10
11     eventPublisher.publishEvent(new VulnerabilityEvent.VulnerabilityDeleted(cveId));
12     return new Result.Success<>(new MessageResponseDTO("Vulnerability deleted successfully"));
13 }

```

Figure 6.3: Service implementation of deleteVulnerability

POST /api/vulnerabilities/

Creates a new vulnerability entry and stores it in both the relational and graph databases.

```

1 public Result<MessageResponseDTO> addNewVulnerability(VulnerabilityDTO vulnerabilityDTO) {
2     if (vulnerabilityRepository.existsByCveId(vulnerabilityDTO.getCveId())) {
3         return new Result.Failure<>(new DomainError.AlreadyExists("Vulnerability " + vulnerabilityDTO.getCveId() + " already exists"));
4     }
5
6     Vulnerability vulnerability = new Vulnerability(vulnerabilityDTO);
7     vulnerabilityRepository.save(vulnerability);
8
9     eventPublisher.publishEvent(new VulnerabilityEvent.VulnerabilityCreated(vulnerability.getCveId(), vulnerability.getDescription(), vulnerability.getBaseScore()));
10    return new Result.Success<>(new MessageResponseDTO("Vulnerability created successfully"));
11 }

```

Figure 6.4: Service implementation of addNewVulnerability

GET /api/vulnerabilities/{CVEID}/packages

Returns a list of all packages and versions that are affected by the specified CVE.

```

1 public Result<List<AffectedVersionDTO>> getPackagesWithVulnerability(String cveId) {
2
3     if (!vulnerabilityGraphRepository.existsByCveId(cveId)) {
4         return new Result.Failure<>(new DomainError.NotFound("Vulnerability " + cveId + " not found"));
5     }
6
7     List<AffectedVersionDTO> affectedVersions = vulnerabilityGraphRepository.findAffectedVersionsByCve(cveId);
8
9     return new Result.Success<>(affectedVersions);
10 }

```

Figure 6.5: Service implementation of getPackagesWithVulnerability

```

1 @Query("""
2     MATCH (vn:Vulnerability {cve_id: $cveId})
3     MATCH (vn)←[:AFFECTED_BY]-(v:Version)
4     RETURN v.package_name AS packageName, collect(v.version) AS versions
5 """)
6 List<AffectedVersionDTO> findAffectedVersionsByCve(@Param("cveId") String cveId);

```

Figure 6.6: Repository implementation of findAffectedVersionsByCve

GET /api/vulnerabilities/transitive/{packageName}/{packageVersion}

Returns all vulnerabilities that affect the specified package indirectly through its dependency tree.

```

1 public Result<List<VulnerabilityReportDTO>> getTransitiveVulnerabilities(String packageName, String packageVersion) {
2
3     if(!packageVersionGraphRepository.existsByPackageNameAndVersion(packageName, packageVersion)) {
4         return new Result.Failure<>(new DomainError.NotFound("Package " + packageName + " " + packageVersion + " not found"));
5     }
6
7     var vulnerabilities = packageVersionGraphRepository.findRecursiveVulnerabilities(packageName, packageVersion);
8     return new Result.Success<>(vulnerabilities);
9 }

```

Figure 6.7: Service implementation of getTransitiveVulnerabilities

```

1 @Query("""
2     MATCH (start:Version {package_name: $package_name, version: $version})
3     MATCH (start)-[:DEPENDS_ON*]->(dep:Version)
4     WITH DISTINCT dep
5     ORDER BY dep.version_array DESC
6     WITH dep.package_name AS pkgName, head(collect(dep)) AS maxVersion
7     MATCH (maxVersion)-[:AFFECTED_BY]->(vuln:Vulnerability)
8     RETURN vuln.cve_id AS vulnerabilityId,
9           pkgName AS affectedPackage,
10          maxVersion.version AS affectedVersion,
11          vuln.baseScore AS baseScore,
12          vuln.description AS description
13 """)
14 List<VulnerabilityReportDTO> findRecursiveVulnerabilities(@Param("package_name") String packageName, @Param("version") String packageVersion);
15

```

Figure 6.8: Repository implementation of findRecursiveVulnerabilities

6.1.2 Projects Controller

POST /api/projects/

Creates a new project owned by the authenticated user.

```

1 public Result<ProjectDTO> addProject(ProjectCreationDTO dto) {
2     if (projectRepository.existsByName(dto.getName())) {
3         return new Result.Failure<>(new DomainError.AlreadyExists("Project " + dto.getName() + " already exists."));
4     }
5
6     String username = getCurrentUsername();
7     Project.Collaborator admin = getCollaboratorInfo(username);
8
9     List<Project.ProjectPackage> packages = dto.getPackages().stream()
10         .map(pkgDto -> new Project.ProjectPackage(pkgDto.getName(), pkgDto.getVersion()))
11         .toList();
12
13     Project project = new Project(dto.getName(), dto.getDescription(), dto.getPythonVersion(), admin, packages);
14     Project savedProject = projectRepository.save(project);
15
16     triggerPostCreationEvents(savedProject);
17     return new Result.Success<>(new ProjectDTO(savedProject));
18 }
19

```

Figure 6.9: Service implementation of addProject

GET /api/projects/{projectName}

Retrieves full details of a specific project.

```
1 public Result<ProjectDTO> getProject(String projectName) {
2     var projectOpt = projectRepository.findByName(projectName);
3
4     if (projectOpt.isEmpty()) {
5         return new Result.Failure<>(new DomainError.NotFound("Project not Found"));
6     }
7
8     Project project = projectOpt.get();
9
10    if (!canView(getCurrentUsername(), project)) {
11        return new Result.Failure<>(new DomainError.AccessDenied("You are not authorized to view this project."));
12    }
13
14    return new Result.Success<>(new ProjectDTO(project));
15 }
```

Figure 6.10: Service implementation of getProject

DELETE /api/projects/{projectName}

Permanently deletes a project. Only the Owner can perform this action.

```
1 public Result<MessageResponseDTO> deleteProject(String projectName) {
2     var projectOpt = projectRepository.findByName(projectName);
3
4     if (projectOpt.isEmpty()) {
5         return new Result.Failure<>(new DomainError.NotFound("Project not found"));
6     }
7
8     Project project = projectOpt.get();
9
10    if (!project.getAdmin().getUsername().equals(getCurrentUsername())) {
11        return new Result.Failure<>(new DomainError.AccessDenied("Only the owner can delete the project"));
12    }
13
14    List<String> userIds = project.getCollaborators().stream()
15        .map(Project.Collaborator::getUsername)
16        .toList();
17
18    projectRepository.delete(project);
19
20    eventPublisher.publishEvent(new ProjectEvent.ProjectDeleted(projectName, userIds));
21    return new Result.Success<>(new MessageResponseDTO("Deleted project " + projectName + "."));
22 }
```

Figure 6.11: Service implementation of deleteProject

PUT /api/projects/{projectName}/users/{collaboratorUsername}

Grants a user access to the project. Only the Owner can add collaborators.

```

1 public Result<MessageResponseDTO> addCollaboratorToProject(String projectName, String collaboratorUsername) {
2     var projectOpt = projectRepository.findByName(projectName);
3     if (projectOpt.isEmpty()) return new Result.Failure<>(new DomainError.NotFound("Project not found"));
4     Project project = projectOpt.get();
5
6     if (!project.getAdmin().getUsername().equals(getCurrentUsername())) {
7         return new Result.Failure<>(new DomainError.AccessDenied("Only owner can add collaborators"));
8     }
9
10    boolean alreadyExists = project.getAdmin().getUsername().equals(collaboratorUsername) ||
11        project.getCollaborators().stream().anyMatch(c -> c.getUsername().equals(collaboratorUsername));
12
13    if (alreadyExists) {
14        return new Result.Failure<>(new DomainError.InvalidOperation("User is already associated with this project"));
15    }
16
17    Project.Collaborator newCollab;
18    try {
19        newCollab = getCollaboratorInfo(collaboratorUsername);
20    } catch (IllegalStateException e) {
21        return new Result.Failure<>(new DomainError.NotFound(e.getMessage()));
22    }
23
24    project.getCollaborators().add(newCollab);
25    project.setLastUpdate(Instant.now());
26
27    projectRepository.save(project);
28
29    eventPublisher.publishEvent(new ProjectEvent.CollaboratorAdded(projectName, collaboratorUsername));
30    return new Result.Success<>(new MessageResponseDTO("Collaborator " + collaboratorUsername + " added"));
31 }

```

Figure 6.12: Service implementation of addCollaboratorToProject

DELETE /api/projects/{projectName}/users/{collaboratorUsername}

Revokes access for a collaborator. Only the Owner can remove collaborators.

```

1 public Result<MessageResponseDTO> removeCollaboratorFromProject(String projectName, String collaboratorUsername) {
2     var projectOpt = projectRepository.findByName(projectName);
3     if (projectOpt.isEmpty()) return new Result.Failure<>(new DomainError.NotFound("Project not found"));
4     Project project = projectOpt.get();
5
6     String currentOwner = project.getAdmin().getUsername();
7     String requester = getCurrentUsername();
8
9     boolean isOwnerRemoving = currentOwner.equals(requester);
10    boolean isSelfRemoving = requester.equals(collaboratorUsername);
11
12    if (!isOwnerRemoving && !isSelfRemoving) {
13        return new Result.Failure<>(new DomainError.AccessDenied("You don't have permission to remove this collaborator"));
14    }
15
16    if (currentOwner.equals(collaboratorUsername)) {
17        return new Result.Failure<>(new DomainError.InvalidOperation("Owner cannot be removed. Transfer ownership."));
18    }
19
20    boolean removed = project.getCollaborators().removeIf(c -> c.getUsername().equals(collaboratorUsername));
21
22    if (removed) {
23        project.setLastUpdate(Instant.now());
24        projectRepository.save(project);
25
26        eventPublisher.publishEvent(new ProjectEvent.CollaboratorRemoved(projectName, collaboratorUsername));
27        return new Result.Success<>(new MessageResponseDTO("Removed collaborator " + collaboratorUsername + " from project " + projectName + "."));
28    }
29
30    return new Result.Failure<>(new DomainError.NotFound("Collaborator not found"));
31 }

```

Figure 6.13: Service implementation of removeCollaboratorFromProject

GET /api/projects/{projectName}/users

Gets the list of all collaborators for the project.

```
1 public Result<List<CollaboratorDTO>> getProjectCollaborators(String projectName) {
2
3     var projectOpt = projectRepository.findByName(projectName);
4     if (projectOpt.isEmpty()) {
5         return new Result.Failure<>(new DomainError.NotFound("Project with name " + projectName + " not found."));
6     }
7
8     String username = getCurrentUsername();
9     var project = projectOpt.get();
10
11     boolean isOwner = project.getAdmin().getUsername().equals(username);
12     boolean isCollaborator = project.getCollaborators().stream()
13         .anyMatch(c -> c.getUsername().equals(username));
14
15     if (!isOwner && !isCollaborator) {
16         return new Result.Failure<>(new DomainError.AccessDenied("You are not authorized to view this project's collaborators."));
17     }
18
19     List<CollaboratorDTO> dtos = project.getCollaborators().stream()
20         .map(CollaboratorDTO::new)
21         .toList();
22
23     return new Result.Success<>(dtos);
24 }
```

Figure 6.14: Service implementation of getProjectCollaborators

DELETE /api/projects/{projectName}/users/me

Allows the authenticated user to voluntarily leave the project.

```
1 public Result<MessageResponseDTO> leaveProject(String projectName) {
2     String requester = getCurrentUsername();
3     return removeCollaboratorFromProject(projectName, requester);
4 }
```

Figure 6.15: Service implementation of leaveProject

PUT /api/projects/{projectName}/packages

Adds new packages or updates the versions of existing ones in the project.

```

1 public Result<MessageResponseDTO> updateProjectPackages(String projectName, List<InstalledPackageDTO> newPackages) {
2     if (newPackages == null || newPackages.isEmpty()) {
3         return new Result.Failure<>(new DomainError.ValidationFailed("Package list cannot be null or empty."));
4     }
5
6     var projectOpt = projectRepository.findByName(projectName);
7     if (projectOpt.isEmpty()) {
8         return new Result.Failure<>(new DomainError.NotFound("Project not found"));
9     }
10    Project project = projectOpt.get();
11
12    if (!canEdit(getCurrentUsername(), project)) {
13        return new Result.Failure<>(new DomainError.AccessDenied("Permission denied"));
14    }
15
16    List<String> missingPackages = new ArrayList<>();
17    for (InstalledPackageDTO pkg : newPackages) {
18        if (packageService.getPackageByNameVersion(pkg.getName(), pkg.getVersion()) instanceof Result.Failure) {
19            missingPackages.add(pkg.getName() + "@" + pkg.getVersion());
20        }
21    }
22
23    if (!missingPackages.isEmpty()) {
24        return new Result.Failure<>(new DomainError.ValidationFailed("The following packages are not in the system: " + String.join(", ", missingPackages)));
25    }
26
27    Set<String> newPackageNames = newPackages.stream()
28        .map(InstalledPackageDTO::getName)
29        .collect(Collectors.toSet());
30
31    project.getPackages().removeIf(p -> newPackageNames.contains(p.getName()));
32
33    List<Project.ProjectPackage> pkgsToAdd = newPackages.stream()
34        .map(dto -> new Project.ProjectPackage(dto.getName(), dto.getVersion()))
35        .toList();
36
37    project.getPackages().addAll(pkgsToAdd);
38    project.setLastUpdate(Instant.now());
39
40    Project savedProject = projectRepository.save(project);
41
42    triggerPostPackageUpdateEvents(savedProject);
43
44    return new Result.Success<>(new MessageResponseDTO("Packages updated for " + projectName + "."));
45 }

```

Figure 6.16: Service implementation of updateProjectPackages

DELETE /api/projects/{projectName}/packages

Removes packages from the project configuration.

```

1 public Result<MessageResponseDTO> removePackagesFromProject(String projectName, List<String> packagesToRemove) {
2     var projectOpt = projectRepository.findByName(projectName);
3     if (projectOpt.isEmpty()) {
4         return new Result.Failure<>(new DomainError.NotFound("Project not found"));
5     }
6     Project project = projectOpt.get();
7
8     if (!canEdit(getCurrentUsername(), project)) {
9         return new Result.Failure<>(new DomainError.AccessDenied("Permission denied"));
10    }
11
12    boolean changed = project.getPackages().removeIf(p ->
13        packagesToRemove.contains(p.getName())
14    );
15
16    if (!changed) {
17        return new Result.Failure<>(new DomainError.NotFound("No packages matched for removal."));
18    }
19
20    project.setLastUpdate(Instant.now());
21    projectRepository.save(project);
22
23    triggerPostPackageUpdateEvents(project);
24
25    return new Result.Success<>(new MessageResponseDTO("Packages removed successfully."));
26 }

```

Figure 6.17: Service implementation of removePackagesFromProject

PUT /api/projects/{projectName}/admin/{newAdminUsername}

Transfers the project admin role to another existing collaborator.

```

1 public Result<MessageResponseDTO> transferOwnership(String projectName, String newAdminUsername) {
2     var projectOpt = projectRepository.findByName(projectName);
3     if (projectOpt.isEmpty()) return new Result.Failure<>(new DomainError.NotFound("Project not found"));
4     Project project = projectOpt.get();
5
6     if (!project.getAdmin().getUsername().equals(getCurrentUsername())) {
7         return new Result.Failure<>(new DomainError.AccessDenied("Only the current owner can transfer ownership"));
8     }
9
10    boolean isCollaborator = project.getCollaborators().stream()
11        .anyMatch(c -> c.getUsername().equals(newAdminUsername));
12
13    if (!isCollaborator) {
14        return new Result.Failure<>(new DomainError.InvalidOperation("New owner must be an existing collaborator"));
15    }
16
17    Project.Collaborator newAdmin = project.getCollaborators().stream()
18        .filter(c -> c.getUsername().equals(newAdminUsername))
19        .findFirst().get();
20
21    project.setAdmin(newAdmin);
22
23    project.setLastUpdate(Instant.now());
24    projectRepository.save(project);
25
26    return new Result.Success<>(new MessageResponseDTO("Ownership transferred to " + newAdminUsername));
27 }

```

Figure 6.18: Service implementation of transferOwnership

6.1.3 Administrator Controller**PUT /api/admin/addNewAdmin/{username}**

Promotes an existing user to have administrator privileges.

```

1 public Result<MessageResponseDTO> addNewAdmin(String username) {
2     var userOpt = userRepository.findByUsername(username);
3     if (userOpt.isEmpty()) {
4         return new Result.Failure<>(new DomainError.NotFound("User " + username + " does not exist"));
5     }
6
7     var user = userOpt.get();
8
9     if ("ROLE_ADMIN".equals(user.getRole())) {
10        return new Result.Failure<>(new DomainError.AlreadyExists("User " + username + " is already an administrator"));
11    }
12
13    user.setRole("ROLE_ADMIN");
14    userRepository.save(user);
15
16    return new Result.Success<>(new MessageResponseDTO("Administrator added successfully"));
17 }

```

Figure 6.19: Service implementation of addNewAdmin

DELETE /api/admin/removeAdmin/{username}

Revokes administrator privileges from an existing administrator.

```

1 public Result<MessageResponseDTO> removeAdmin(String username) {
2     var userOpt = userRepository.findByUsername(username);
3     if (userOpt.isEmpty()) {
4         return new Result.Failure<>(new DomainError.NotFound("User " + username + " does not exist"));
5     }
6
7     var user = userOpt.get();
8     if ("ROLE_USER".equals(user.getRole())) {
9         return new Result.Failure<>(new DomainError.AlreadyExists("User " + username + " is already a standard user"));
10    }
11
12    user.setRole("ROLE_USER");
13    userRepository.save(user);
14
15    return new Result.Success<>(new MessageResponseDTO("Administrator's privilege removed successfully"));
16 }

```

Figure 6.20: Service implementation of removeAdmin

GET /api/admin/packagesWithPageRank/{limit}

Returns the list of packages ranked by PageRank score, representing their global influence across the dependency network.

```

1 public Result<List<PageRankResultDTO>> getTopByPageRank(Integer limit) {
2     var results = PackageGraphRepository.topByPageRank(limit);
3     return new Result.Success<>(results);
4 }

```

Figure 6.21: Service implementation of getTopByPageRank

```

1 @Query("""
2     CALL gds.pageRank.stream('pkgGraph')
3     YIELD nodeId, score
4     WITH gds.util.asNode(nodeId) AS v, score
5     RETURN v.package_name AS package_name, v.version AS version, score
6     ORDER BY score DESC
7     LIMIT $limit
8 """)
9 List<PageRankResultDTO> topByPageRank(@Param("limit") Integer limit);

```

Figure 6.22: Repository implementation of topByPageRank

GET /api/admin/packagesWithHighCentrality

Returns the list of packages that are depended on by the largest number of other packages (Degree Centrality).

```
1 public Result<List<CentralityResultDTO>> getTopByDegree() {
2     var results = PackageGraphRepository.topByDegree();
3     return new Result.Success<>(results);
4 }
```

Figure 6.23: Service implementation of getTopByDegree

```
1 @Query("""
2     MATCH (v:Version)←[:DEPENDS_ON]-(d:Version)
3     WITH v.package_name AS package_name, count(DISTINCT d) AS score
4     RETURN package_name, score
5     ORDER BY score DESC
6     LIMIT 10
7 """)
8 List<CentralityResultDTO> topByDegree();
```

Figure 6.24: Repository implementation of topByDegree

6.1.4 Package Controller

GET /api/packages/{packageName}

Returns all information about the latest version of the specified package, including metadata, dependencies and vulnerabilities.

```
1 public Result<PackageVersionDTO> getPackageByName(String packageName) {
2     var optionalPackage = packageVersionRepository.findTopByPackageNameOrderByVersionArrayDesc(packageName);
3
4     if (optionalPackage.isPresent()) {
5         return new Result.Success<>(new PackageVersionDTO(optionalPackage.get()));
6     }
7
8     return new Result.Failure<>(new DomainError.NotFound("Package " + packageName + " not found."));
9 }
```

Figure 6.25: Service implementation of getPackageByName

PUT /api/packages/{packageName}

Updates metadata for all versions of the specified package.

```

1 public Result<MessageResponseDTO> updatePackageMetadata(String packageName, UpdateGeneralPackageDTO updateData) {
2     if (!packageVersionRepository.existsByPackageName(packageName)) {
3         return new Result.Failure<>(new DomainError.NotFound("Package " + packageName + " not found."));
4     }
5
6     helper.updatePackageGeneralMetadata(packageName, updateData);
7
8     updateData.getDocumentationURL().ifPresent(url → eventPublisher.publishEvent(new PackageEvent.UpdateDocumentation(packageName, url)));
9     return new Result.Success<>(new MessageResponseDTO("Update executed successfully"));
10 }

```

Figure 6.26: Service implementation of updatePackageMetadata

GET /api/packages/{packageName}/{version}

Returns all information about a specific version of a package.

```

1 public Result<PackageVersionDTO> getPackageByNameVersion(String packageName, String packageVersion) {
2     var optionalPackage = packageVersionRepository.findByPackageNameAndVersion(packageName, packageVersion);
3
4     if (optionalPackage.isPresent()) {
5         return new Result.Success<>(new PackageVersionDTO(optionalPackage.get()));
6     }
7
8     return new Result.Failure<>(new DomainError.NotFound("Version " + packageVersion + " of package " + packageName + " not found."));
9 }

```

Figure 6.27: Service implementation of getPackageByNameVersion

PUT /api/packages/{packageName}/{version}

Updates details of a specific version, such as dependencies, vulnerabilities and Python requirements.

```

1 public Result<MessageResponseDTO> updatePackageVersion(String packageName, String version, UpdatePackageVersionDTO updateVersionDTO) {
2     var existingOpt = packageVersionRepository.findByPackageNameAndVersion(packageName, version);
3     if (existingOpt.isEmpty()) {
4         return new Result.Failure<>(new DomainError.NotFound("Package " + packageName + " " + version + " not found."));
5     }
6
7     PackageVersion updateVersion = existingOpt.get();
8     updateVersionDTO.getRequiresPython().ifPresent(updateVersion::setRequiresPython);
9     updateVersion.setUploadTime(Instant.now());
10
11     if (updateVersionDTO.getDependencies() != null) {
12         List<Constraints> dependencyList = updateVersionDTO.getDependencies().stream()
13             .map(DependencyParser::parseFullString)
14             .toList();
15         updateVersion.setDependencies(dependencyList);
16     }
17
18     if (updateVersionDTO.getVulnerabilities() != null) {
19         List<EmbeddedVulnerability> vulnList = updateVersionDTO.getVulnerabilities().stream()
20             .map(EmbeddedVulnerability::new)
21             .toList();
22         updateVersion.setVulnerabilities(vulnList);
23     }
24
25     packageVersionRepository.save(updateVersion);
26
27     List<ConstraintsDTO> constraintsDtos = (updateVersion.getDependencies() != null)
28         ? updateVersion.getDependencies().stream().map(ConstraintsDTO::new).toList()
29         : new ArrayList<>();
30     eventPublisher.publishEvent(new PackageEvent.UpdatePackageVersion(packageName, version, constraintsDtos, updateVersionDTO.getVulnerabilities()));
31
32     return new Result.Success<>(new MessageResponseDTO("Package updated successfully"));
33 }

```

Figure 6.28: Service implementation of updatePackageVersion

DELETE /api/packages/{packageName}/{version}

Deletes a specific version from the database and graph.

```
1 public Result<MessageResponseDTO> deletePackageVersion(String packageName, String version) {
2     if (!packageVersionRepository.existsByPackageNameAndVersion(packageName, version)) {
3         return new Result.Failure<>(new DomainError.NotFound("Package version not found."));
4     }
5
6     packageVersionRepository.deleteByPackageNameAndVersion(packageName, version);
7
8     eventPublisher.publishEvent(new PackageEvent.DeletePackageVersion(packageName, version));
9     return new Result.Success<>(new MessageResponseDTO("Package deleted successfully"));
10 }
```

Figure 6.29: Service implementation of deletePackageVersion

POST /api/packages/{packageName}/version

Creates a new version of a package in the database and updates graph relationships.

```
1 public Result<MessageResponseDTO> addNewVersion(String packageName, AddPackageVersionDTO newVersionDTO) {
2     if (!packageName.equals(newVersionDTO.getPackageName())) {
3         return new Result.Failure<>(new DomainError.InvalidOperation("The package name in the URL does not match the body"));
4     }
5
6     String version = newVersionDTO.getVersion();
7     if (packageVersionRepository.existsByPackageNameAndVersion(packageName, version)) {
8         return new Result.Failure<>(new DomainError.AlreadyExists("Version " + version + " already exists."));
9     }
10
11     PackageVersion versionDoc = new PackageVersion(newVersionDTO, versionParser);
12     packageVersionRepository.save(versionDoc);
13
14     PublishedVersionDTO publishedVersionDTO = new PublishedVersionDTO(versionDoc);
15     eventPublisher.publishEvent(new PackageEvent.VersionRelease(publishedVersionDTO));
16
17     return new Result.Success<>(new MessageResponseDTO("Package version created successfully"));
18 }
```

Figure 6.30: Service implementation of addNewVersion

GET /api/packages/{packageName}/{version}/reverse-dependencies

Returns a list of package versions that depend on the specified package version.

```

1 public Result<List<ReverseDependencyDTO>> getPackagesDependingOn(String packageName, String version) {
2     if (!packageVersionGraphRepository.existsByPackageNameAndVersion(packageName, version)) {
3         return new Result.Failure<>(new DomainError.NotFound("Package " + packageName + " not found in the system."));
4     }
5
6     List<PackageVersionNode> dependents = packageVersionGraphRepository.findReverseDependencies(packageName, version);
7
8     Map<String, List<String>> grouped = new HashMap<>();
9
10    for (PackageVersionNode node : dependents) {
11        String name = node.getPackageName();
12        String ver = node.getVersion();
13        grouped.computeIfAbsent(name, k → new ArrayList<>());
14        grouped.get(name).add(ver);
15    }
16
17    List<ReverseDependencyDTO> result = new ArrayList<>();
18    for (Map.Entry<String, List<String>> entry : grouped.entrySet()) {
19        result.add(new ReverseDependencyDTO(entry.getKey(), entry.getValue()));
20    }
21
22    return new Result.Success<>(result);
23 }

```

Figure 6.31: Service implementation of getPackagesDependingOn

```

1 @Query("""
2     MATCH (target:Version {package_name: $packageName, version: $version})
3     MATCH (dependent:Version)-[:DEPENDS_ON]-(target)
4     RETURN dependent
5 """)
6 List<PackageVersionNode> findReverseDependencies(@Param("packageName") String packageName, @Param("version") String version);
7

```

Figure 6.32: Repository implementation of findReverseDependencies

GET /api/packages/{packageName}/{version}/direct-dependencies

Returns a list of direct dependencies for a specific package version.

```

1 public Result<List<String>> getDirectDependencies(String packageName, String version) {
2     var versionDocOpt = packageVersionRepository.findByPackageNameAndVersion(packageName, version);
3
4     if (versionDocOpt.isEmpty()) {
5         return new Result.Failure<>(new DomainError.NotFound("Package " + packageName + " version " + version + " not found in database."));
6     }
7
8     List<Constraints> rawDependencies = versionDocOpt.get().getDependencies();
9     if (rawDependencies == null) {
10        rawDependencies = new ArrayList<>();
11    }
12
13    List<String> result = new ArrayList<>();
14    for (Constraints dependency: rawDependencies) {
15        result.add(dependency.getFull());
16    }
17
18    return new Result.Success<>(result);
19 }

```

Figure 6.33: Service implementation of getDirectDependencies

GET /api/packages/{packageName}/safe

Returns the last version of the specified package with no known vulnerabilities.

```

1 public Result<PackageVersionDTO> getSafeVersions(String packageName) {
2     if (!packageVersionRepository.existsByPackageName(packageName)) {
3         return new Result.Failure<>(new DomainError.NotFound("Package " + packageName + " not found."));
4     }
5
6     var optionalSafeVersion = packageVersionRepository.findTopByPackageNameAndRiskScoreOrderByVersionArrayDesc(packageName, 0);
7
8     PackageVersionDTO safeVersion = new PackageVersionDTO(optionalSafeVersion.get());
9
10    return new Result.Success<>(safeVersion);
11 }

```

Figure 6.34: Service implementation of getSafeVersions

GET /api/packages/{packageName}/indirect-safe

Returns the latest version of the specified package with no known vulnerabilities, also taking transitive vulnerabilities into account.

```

1 public Result<?> getIndirectSafeVersions(String packageName) {
2     if (!packageVersionRepository.existsByPackageName(packageName)) {
3         return new Result.Failure<>(new DomainError.NotFound("Package " + packageName + " not found."));
4     }
5
6     var safeVersions = packageVersionRepository.findByPackageNameAndRiskScoreOrderByVersionArrayDesc(packageName, 0);
7
8     for (PackageVersion version : safeVersions) {
9         List<VulnerabilityReportDTO> vulnerabilities = packageVersionGraphRepository.findRecursiveVulnerabilities(version.getPackageName(), version.getVersion());
10        if (vulnerabilities.isEmpty()) {
11            return new Result.Success<>(new PackageVersionDTO(version));
12        }
13    }
14    return new Result.Success<>(new MessageResponseDTO("No versions found."));
15 }

```

Figure 6.35: Service implementation of getIndirectSafeVersions

```

1 @Query("""
2     MATCH (start:Version {package_name: $package_name, version: $version})
3     MATCH (start)-[:DEPENDS_ON*]->(dep:Version)
4     WITH DISTINCT dep
5     ORDER BY dep.version_array DESC
6     WITH dep.package_name AS pkgName, head(collect(dep)) AS maxVersion
7     MATCH (maxVersion)-[:AFFECTED_BY]->(vuln:Vulnerability)
8     RETURN vuln.cve_id AS vulnerabilityId,
9            pkgName AS affectedPackage,
10            maxVersion.version AS affectedVersion,
11            vuln.baseScore AS baseScore,
12            vuln.description AS description
13 """)
14 List<VulnerabilityReportDTO> findRecursiveVulnerabilities(@Param("package_name") String packageName, @Param("version") String packageVersion);
15

```

Figure 6.36: Repository implementation of findRecursiveVulnerabilities

6.1.5 Authentication Controller

POST /api/auth/register

Creates a new user account and returns a JWT token.

```
1 public Result<AuthResponseDTO> register(RegisterRequestDTO request) {
2
3     if (userRepository.existsByUsername(request.getUsername())) {
4         return new Result.Failure<>(new DomainError.AlreadyExists("Username is already taken"));
5     }
6
7     if (userRepository.existsByEmail(request.getEmail())) {
8         return new Result.Failure<>(new DomainError.AlreadyExists("Email already in use"));
9     }
10
11     User user = new User(request);
12     user.setPassword(passwordEncoder.encode(request.getPassword()));
13     user.setRole("ROLE_USER");
14     user.setProjectNames(new ArrayList<>());
15
16     try {
17         User savedUser = userRepository.save(user);
18         log.info("User saved in MongoDB: {}", savedUser.getUsername());
19
20         String token = jwtUtil.generateToken(savedUser.getId(), savedUser.getUsername(), savedUser.getRole());
21
22         return new Result.Success<>(new AuthResponseDTO(token, savedUser.getUsername(), savedUser.getEmail()));
23     } catch (Exception e) {
24         log.error("Failed to save user in Mongo.", e);
25         return new Result.Failure<>(new DomainError.SystemError(e));
26     }
27 }
```

Figure 6.37: Service implementation of register

POST /api/auth/login

Authenticates a user and returns a JWT token.

```
1 public Result<AuthResponseDTO> login(LoginRequestDTO request) {
2
3     var userOpt = userRepository.findByUsername(request.getUsername());
4     if (userOpt.isEmpty()) {
5         return new Result.Failure<>(new DomainError.InvalidCredentials("Invalid username or password"));
6     }
7
8     User user = userOpt.get();
9     if (!passwordEncoder.matches(request.getPassword(), user.getPassword())) {
10         return new Result.Failure<>(new DomainError.InvalidCredentials("Invalid username or password"));
11     }
12
13     String token = jwtUtil.generateToken(user.getId(), user.getUsername(), user.getRole());
14
15     return new Result.Success<>(new AuthResponseDTO(token, user.getUsername(), user.getEmail()));
16 }
```

Figure 6.38: Service implementation of login

6.1.6 User Controller

GET /api/users/me

Retrieves the profile information of the currently authenticated user.

```
1 public Result<UserDTO> getProfile(String username) {  
2     log.info("Get user profile");  
3  
4     var optUser = userRepository.findByUsername(username);  
5     if (optUser.isEmpty()) {  
6         return new Result.Failure<>(new DomainError.NotFound("User not found"));  
7     }  
8  
9     User user = optUser.get();  
10    return new Result.Success<>(new UserDTO(user));  
11 }
```

Figure 6.39: Service implementation of getProfile

PUT /api/users/me

Updates the current user's profile information.

```

1  public Result<UserDTO> updateProfile(String username, UpdateProfileDTO request) {
2      var userOpt = userRepository.findByUsername(username);
3      if (userOpt.isEmpty()) {
4          return new Result.Failure<>(new DomainError.NotFound("User not found"));
5      }
6
7      User user = userOpt.get();
8      boolean importantChanges = false;
9      boolean isUpdated = false;
10
11     if (request.getEmail() != null && !request.getEmail().isBlank() &&
12         !request.getEmail().equals(user.getEmail())) {
13
14         if (userRepository.existsByEmail(request.getEmail())) {
15             return new Result.Failure<>(new DomainError.AlreadyExists("Email already taken"));
16         }
17
18         user.setEmail(request.getEmail());
19         isUpdated = true;
20         importantChanges = true;
21     }
22
23     if (request.getFirstName() != null && !request.getFirstName().isBlank()) {
24         user.setFirstName(request.getFirstName());
25         isUpdated = true;
26     }
27
28     if (request.getLastName() != null && !request.getLastName().isBlank()) {
29         user.setLastName(request.getLastName());
30         isUpdated = true;
31     }
32
33     if (request.getPassword() != null && !request.getPassword().isBlank()) {
34         user.setPassword(passwordEncoder.encode(request.getPassword()));
35         isUpdated = true;
36     }
37
38     if (!isUpdated) {
39         return new Result.Success<>(new UserDTO(user));
40     }
41
42     User savedUser = userRepository.save(user);
43
44     if (importantChanges && savedUser.getProjectNames() != null && !savedUser.getProjectNames().isEmpty()) {
45         eventPublisher.publishEvent(new UserEvent.UserUpdated(
46             savedUser.getProjectNames(),
47             username,
48             savedUser.getUsername(),
49             savedUser.getEmail()
50         ));
51     }
52
53     return new Result.Success<>(new UserDTO(savedUser));
54 }

```

Figure 6.40: Service implementation of updateProfile

DELETE /api/users/me

Permanently deletes the authenticated user's account and all associated data.

```
1 public Result<MessageResponseDTO> deleteUser(String username) {
2     var userOpt = userRepository.findByUsername(username);
3     if (userOpt.isEmpty()) {
4         return new Result.Failure<>(new DomainError.NotFound("User not found."));
5     }
6
7     User user = userOpt.get();
8     if (!user.getProjectNames().isEmpty()) {
9         return new Result.Failure<>(new DomainError.InvalidOperation("Quit from all your projects before delete your account."));
10    }
11
12    userRepository.deleteByUsername(username);
13    return new Result.Success<>(new MessageResponseDTO("User profile deleted."));
14 }
```

Figure 6.41: Service implementation of deleteUser

GET /api/users/me/projects

Retrieves a list of project names that the current user is involved in.

```
1 public Result<List<String>> getUserProjectNames(String username) {
2     log.info("Get user projects");
3
4     var optUser = userRepository.findByUsername(username);
5     if (optUser.isEmpty()) {
6         return new Result.Failure<>(new DomainError.NotFound("User not found"));
7     }
8
9     User user = optUser.get();
10    List<String> projectNames = user.getProjectNames() != null ? user.getProjectNames() : List.of();
11
12    return new Result.Success<>(projectNames);
13 }
```

Figure 6.42: Service implementation of getUserProjectNames

6.2 MongoDB Aggregation

The following set of aggregation queries constitutes the core of the System Administration Dashboard. Designed exclusively for the platform administrator, these analytics provide a holistic view of the application's ecosystem. Through dedicated private interfaces, the administrator can monitor the health and security posture of the repository, including the distribution of risk scores across Python packages and the volume of developer-managed projects. Additionally, these metrics offer vital statistics regarding the registered user base, allowing the administrator to assess the overall integrity and growth trends of the system.

6.2.1 Packages Risk Score Analysis

The `aggregateRiskBuckets` method executes a statistical aggregation on the MongoDB `package` collection. The primary objective of this query is to classify software packages into discrete risk intervals (buckets) based on their computed `risk_score`, considering only the most recent version of each package to ensure data relevance.

The aggregation pipeline handles data deduplication (sort + group + first operations) and statistical binning in a single execution context. The process is structured into the following stages:

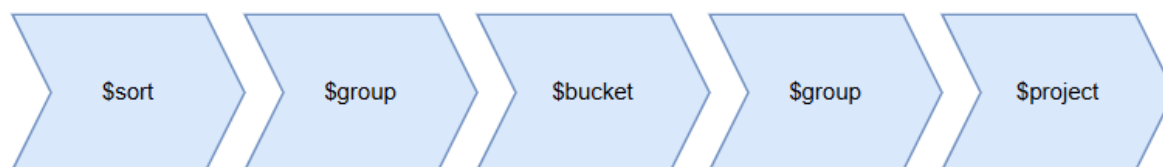


Figure 6.43: Illustration of the aggregation pipeline for risk distribution analysis.

1. **Sorting (sort):** Orders the documents using a compound key: first by `package_name` in ascending order and secondarily by `version_array` in descending order. This specific order is critical to ensure that when grouping by package name in the next stage, the first document processed is always the latest version.
2. **Deduplication (group):** Groups documents by `package_name`. By utilizing the `first` operator on the `risk_score` field, the pipeline retains only the risk score associated with the top-sorted document (the latest version), effectively filtering out obsolete package versions.
3. **Bucketing (bucket):** Categorizes the deduplicated packages into specific risk intervals defined by the boundaries: `[0, 2)`, `[2, 4)`, `[4, 6)`, `[6, 8)`, `[8, 10)`. Any value falling outside these ranges (e.g., ≥ 10) is assigned to a default "Other" bucket. This stage computes the count of packages for each interval.
4. **Global Aggregation (group):** Re-groups the output of the bucketing stage into a single document. It calculates the `totalPackages` (sum of all counts) and constructs a `buckets` array containing the detailed distribution data.
5. **Projection (project):** Finalizes the output structure by excluding technical identifiers (such as `_id`) and retaining only the computed statistics required by the DTO.

Sample Result The query result is mapped directly to the `RiskAggregationDTO` class. The JSON structure below illustrates the hierarchical organization of the response, containing the total count and the breakdown by risk interval:

```
{
```

```
"totalPackages": 150,
"buckets": [
  {"riskInterval": 0, "count": 45},
  {"riskInterval": 2, "count": 30},
  {"riskInterval": 4, "count": 20},
  {"riskInterval": 6, "count": 38},
  {"riskInterval": 8, "count": 12},
  {"riskInterval": "Other", "count": 5}
]
}
```

Method Implementation Below is the reference to the Java implementation of the `aggregateRiskBuckets` method, located in the `PackageDAO` repository class.

```
1 public RiskAggregationDTO aggregateRiskBuckets() {
2     Aggregation aggregation = Aggregation.newAggregation(
3         Aggregation.sort(Sort.by(Sort.Direction.ASC, "package_name")
4             .and(Sort.by(Sort.Direction.DESC, "version_array"))),
5
6         Aggregation.group("package_name")
7             .first("package_name").as("package_name")
8             .first("risk_score").as("risk_score"),
9
10        Aggregation.bucket("risk_score")
11            .withBoundaries(0, 2, 4, 6, 8, 10)
12            .withDefaultBucket("Other")
13            .andOutputCount().as("count"),
14
15        Aggregation.group().sum("count").as("totalPackages")
16            .push(new Document("riskInterval", "$_id").append("count", "$count")).as("buckets"),
17
18        Aggregation.project().andExclude("_id")
19            .andInclude("totalPackages", "buckets")
20    );
21
22    AggregationResults<RiskAggregationDTO> result = mongoTemplate.aggregate(aggregation, "package", RiskAggregationDTO.class);
23
24    return result.getUniqueMappedResult();
25 }
```

Figure 6.44: Method implementation of `aggregateRiskBuckets`

MongoDB Shell Implementation The following script represents the raw aggregation query used to compute risk distribution. It demonstrates the use of the `$bucket` stage for statistical binning and a two-step grouping process for deduplication and summarization.

```

1  [
2    {
3      "$sort": {
4        "package_name": 1,
5        "version_array": -1
6      }
7    },
8    {
9      "$group": {
10       "_id": "$package_name",
11       "package_name": { "$first": "$package_name" },
12       "risk_score": { "$first": "$risk_score" }
13     }
14   },
15   {
16     "$bucket": {
17       "groupBy": "$risk_score",
18       "boundaries": [0, 2, 4, 6, 8, 10],
19       "default": "Other",
20       "output": {
21         "count": { "$sum": 1 }
22       }
23     }
24   },
25   {
26     "$group": {
27       "_id": null,
28       "totalPackages": { "$sum": "$count" },
29       "buckets": {
30         "$push": {
31           "riskInterval": "$_id",
32           "count": "$count"
33         }
34       }
35     }
36   },
37   {
38     "$project": {
39       "_id": 0,
40       "totalPackages": 1,
41       "buckets": 1
42     }
43   }
44 ]

```

Figure 6.45: Raw MongoDB query of `aggregateRiskBuckets`

6.2.2 Monthly Vulnerability Trend Analysis

The `getVulnerabilityTrendLastYear` method executes a temporal aggregation query on the MongoDB `vulnerability` collection. This analysis is critical for monitoring the influx of new security threats over a rolling 12-month window. By grouping vulnerabilities by their publication date, the system provides the administrator with a clear visualization of threat trends and seasonal spikes.

The aggregation pipeline optimizes performance by filtering the dataset before processing the temporal components. The process is structured into the following stages:

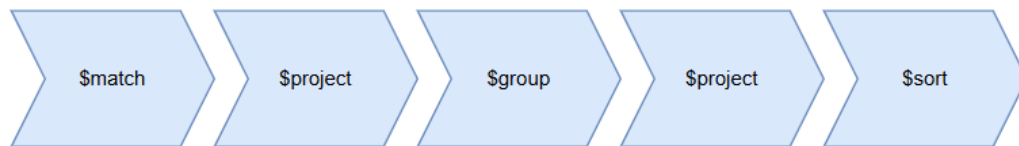


Figure 6.46: Illustration of the aggregation pipeline for vulnerability trend analysis.

1. **Temporal Filtering (match):** Filters the document set to retain only vulnerabilities published within the last 12 months. Placing this filter at the beginning of the pipeline is crucial for performance, as it discards irrelevant historical data before any transformation occurs. The lower bound is dynamically calculated relative to the execution time (`published ≥ twelveMonthsAgo`).
2. **Component Extraction (project):** Decomposes the `published` date field into granular temporal components, specifically extracting the `year` and `month`. This prepares the data for categorical grouping.
3. **Frequency Aggregation (group):** Groups the documents by the composite key of `year` and `month`, calculating the total count of vulnerabilities reported in each specific time unit.
4. **Result Formatting (project):** Reshapes the output document to match the Data Transfer Object (DTO) structure. It flattens the composite grouping key (extracting `year` and `month` from the `_id` field) and retains the calculated `count`, excluding internal identifiers.
5. **Chronological Sorting (sort):** Orders the resulting dataset in ascending order based on `year` and `month`, ensuring the trend data is presented linearly from the oldest relevant month to the present.

Sample Result The result is returned as a list of `VulnerabilityTrendDTO` objects, representing a time series suitable for charting. The JSON structure below demonstrates the output format:

```
[
  {"year": 2023, "month": 8, "count": 142},
  {"year": 2023, "month": 9, "count": 98},
  {"year": 2023, "month": 10, "count": 215}
]
```

Method Implementation Below is the reference to the Java implementation of the `getVulnerabilityTrendLastYear` method, located in the `VulnerabilityDAO` repository class.

```
1  public List<VulnerabilityTrendDTO> getVulnerabilityTrendLastYear() {
2      LocalDateTime twelveMonthsAgo = LocalDateTime.now().minusMonths(12);
3
4      Aggregation aggregation = Aggregation.newAggregation(
5          Aggregation.match(Criteria.where("published").gte(twelveMonthsAgo)),
6
7          Aggregation.project()
8              .andExpression("year(published)").as("year")
9              .andExpression("month(published)").as("month"),
10
11          Aggregation.group("year", "month").count().as("count"),
12
13          Aggregation.project("count")
14              .and("_id.year").as("year")
15              .and("_id.month").as("month")
16              .andExclude("_id"),
17
18          Aggregation.sort(Sort.Direction.ASC, "year", "month")
19      );
20
21      AggregationResults<VulnerabilityTrendDTO> results = mongoTemplate.aggregate(
22          aggregation, "vulnerability", VulnerabilityTrendDTO.class
23      );
24
25      return results.getMappedResults();
26  }
```

Figure 6.47: Method implementation of `getVulnerabilityTrendLastYear`

MongoDB Shell Implementation The script below illustrates the raw aggregation logic corresponding to the Java implementation.

```

1  [
2    {
3      "$match": { "$expr": { "$gte": [
4        "$published",
5        {
6          "$dateSubtract": {
7            "startDate": "$$NOW",
8            "unit": "month",
9            "amount": 12
10         }
11       }
12     ] } }
13   },
14   {
15     "$project": {
16       "year": { "$year": "$published" },
17       "month": { "$month": "$published" }
18     }
19   },
20   {
21     "$group": {
22       "_id": {
23         "year": "$year",
24         "month": "$month"
25       },
26       "count": { "$sum": 1 }
27     }
28   },
29   {
30     "$project": {
31       "_id": 0,
32       "year": "$_id.year",
33       "month": "$_id.month",
34       "count": 1
35     }
36   },
37   {
38     "$sort": {
39       "year": 1,
40       "month": 1
41     }
42   }
43 ]

```

Figure 6.48: Raw MongoDB Query for vulnerability trends

6.2.3 Critical Threat Identification

The `getMostDangerousVulnerabilities` method performs a targeted risk assessment query on the `vulnerability` collection. Designed to assist administrators in immediate prioritization, this query isolates the most critical threats currently affecting the ecosystem. It specifically targets vulnerabilities that satisfy a set of the most dangerous criteria: remotely exploitable, requiring no authentication, having low access complexity and classified as High or Critical severity.

The aggregation pipeline functions as a high-priority filter, structured into the following stages:



Figure 6.49: Illustration of the aggregation pipeline for identifying most critical vulnerabilities.

1. **Risk Profile Filtering (match):** Applies a strict set of criteria to isolate the most dangerous vectors. The filter selects documents where:
 - `metrics.accessVector` is "NETWORK" (remotely exploitable);
 - `metrics.authentication` is "NONE" (no credentials required);
 - `metrics.accessComplexity` is "LOW" (trivial to exploit);
 - `metrics.baseSeverity` is either "HIGH" or "CRITICAL".
2. **Severity Prioritization (sort):** Orders the filtered vulnerabilities by their `metrics.baseScore` in descending order. This ensures that the threats with the highest CVSS score (closest to 10.0) appear at the top of the list for immediate attention.
3. **Data Projection (project):** Maps the internal database fields to the external API contract. It explicitly renames:
 - `cve_id` → `cveId`
 - `metrics.baseSeverity` → `severity`
 - `metrics.baseScore` → `baseScore`
 - `published` → `publishedDate`

It also excludes technical identifiers like `_id`.
4. **Top-K Selection (limit):** Truncates the result set to a predefined number (defined by the `limit` parameter), providing a concise list of the top N most urgent threats.

Sample Result The result is returned as a list of `PerfectStormVulnerabilityDTO` objects. The JSON structure below illustrates a typical high-severity entry:

```
[
  {
    "cveId": "CVE-2024-3094",
    "description": "Malicious code injection allowing unauthorized remote access.",
    "severity": "CRITICAL",
    "baseScore": 10.0,
    "publishedDate": "2024-03-29"
  },
  {
    "cveId": "CVE-2023-44487",
    "description": "HTTP/2 protocol vulnerability allowing rapid reset attacks (DDoS).",
    "severity": "HIGH",
    "baseScore": 7.5,
    "publishedDate": "2023-10-10"
  }
]
```

Method Implementation Below is the reference to the Java implementation of the `getMostDangerousVulnerabilities` method, located in the `VulnerabilityDAO` repository class.

```
1  public List<PerfectStormVulnerabilityDTO> getMostDangerousVulnerabilities(int limit) {
2
3      Aggregation aggregation = Aggregation.newAggregation(
4          Aggregation.match(
5              Criteria.where("metrics.accessVector").is("NETWORK")
6                  .and("metrics.authentication").is("NONE")
7                  .and("metrics.accessComplexity").is("LOW")
8                  .and("metrics.baseSeverity").in("HIGH", "CRITICAL")
9          ),
10
11          Aggregation.sort(Sort.by(Sort.Direction.DESC, "metrics.baseScore")),
12
13          Aggregation.project()
14              .and("cve_id").as("cveId")
15              .and("description").as("description")
16              .and("metrics.baseSeverity").as("severity")
17              .and("metrics.baseScore").as("baseScore")
18              .and("published").as("publishedDate")
19              .andExclude("_id"),
20
21          Aggregation.limit(limit)
22      );
23
24      AggregationResults<PerfectStormVulnerabilityDTO> results = mongoTemplate.aggregate(
25          aggregation, "vulnerability", PerfectStormVulnerabilityDTO.class
26      );
27
28      return results.getMappedResults();
29  }
30
```

Figure 6.50: Method implementation of `getMostDangerousVulnerabilities`

MongoDB Shell Implementation The following script demonstrates the equivalent raw MongoDB query executed by the system.

```

1  [
2    {
3      "$match": {
4        "metrics.accessVector": "NETWORK",
5        "metrics.authentication": "NONE",
6        "metrics.accessComplexity": "LOW",
7        "metrics.baseSeverity": { "$in": ["HIGH", "CRITICAL"] }
8      }
9    },
10   {
11     "$sort": {
12       "metrics.baseScore": -1
13     }
14   },
15   {
16     "$project": {
17       "_id": 0,
18       "cveId": "$cve_id",
19       "description": "$description",
20       "severity": "$metrics.baseSeverity",
21       "baseScore": "$metrics.baseScore",
22       "publishedDate": "$published"
23     }
24   },
25   {
26     "$limit": 10
27   }
28 ]

```

Figure 6.51: Raw MongoDB Query for critical vulnerability identification

6.2.4 Top Used Packages Analysis

The `mostUsedPackages` method executes a frequency analysis aggregation on the MongoDB collection named `project`. Its primary purpose is to identify the most widely adopted software dependencies across the entire project ecosystem. By counting the occurrences of unique package-version pairs, the system generates a leaderboard of the most popular libraries.

The aggregation pipeline relies heavily on the deconstruction of array fields to perform granular counting. The process is structured into the following stages:

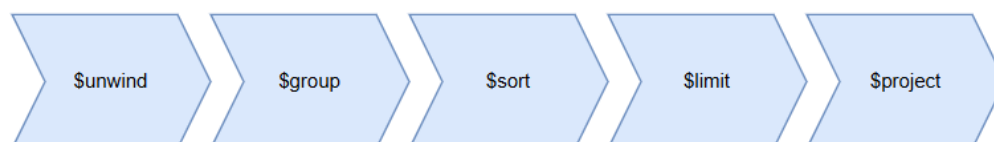


Figure 6.52: Illustration of the aggregation pipeline for top packages analysis.

1. **Deconstruction (unwind):** The initial stage flattens the `packages` array. Since a single project document contains a list of multiple dependencies, this operation replicates the root document for each element in the array, allowing subsequent stages to process each dependency individually.
2. **Frequency Grouping (group):** Groups the documents by a composite key consisting of the package `name` and `version`. It calculates the `count` of occurrences for each unique pair, effectively determining how many projects depend on a specific version of a library.

3. **Ranking (sort):** Sorts the grouped results by the calculated `count` in descending order, placing the most frequently used packages at the top of the dataset.
4. **Top-K Selection (limit):** Restricts the output to the top N results (defined by the `limit` parameter) to focus the analysis on the most significant dependencies.
5. **Projection (project):** Reshapes the final document to align with the DTO structure. It extracts the package name and version from the grouping identifier (`_id`) and retains the usage count, while excluding technical fields.

Sample Result The query returns a list of `AggreagationPackageDTO` objects. The JSON structure below depicts a typical result showing a popular library:

```
[
  {
    "packageName": "react",
    "version": "18.2.0",
    "count": 145
  },
  {
    "packageName": "lodash",
    "version": "4.17.21",
    "count": 89
  }
]
```

Method Implementation Below is the reference to the Java implementation of the `mostUsedPackages` method, located in the `ProjectDAO` repository class.

```
1 public List<AggreagationPackageDTO> mostUsedPackages(int limit) {
2
3     Aggregation aggregation = Aggregation.newAggregation(
4         Aggregation.unwind("packages"),
5
6         Aggregation.group("packages.name", "packages.version").count().as("count"),
7
8         Aggregation.sort(Sort.by(Sort.Direction.DESC, "count")),
9
10        Aggregation.limit(limit),
11
12        Aggregation.project()
13            .and("_id.name").as("packageName")
14            .and("_id.version").as("version")
15            .and("count").as("count")
16    );
17
18    AggregationResults<AggreagationPackageDTO> results = mongoTemplate.aggregate(
19        aggregation, "project", AggreagationPackageDTO.class
20    );
21
22    return results.getMappedResults();
23 }
```

Figure 6.53: Method implementation of `mostUsedPackages`

MongoDB Shell Implementation The following script demonstrates the raw MongoDB aggregation query corresponding to the Java implementation. It highlights the use of `$unwind` to handle the one-to-many relationship between projects and their dependencies.

```

1  [
2    {
3      "$unwind": "$packages"
4    },
5    {
6      "$group": {
7        "_id": {
8          "name": "$packages.name",
9          "version": "$packages.version"
10       },
11       "count": { "$sum": 1 }
12     },
13   },
14   {
15     "$sort": {
16       "count": -1
17     }
18   },
19   {
20     "$limit": 10
21   },
22   {
23     "$project": {
24       "_id": 0,
25       "packageName": "$_id.name",
26       "version": "$_id.version",
27       "count": 1
28     }
29   }
30 ]

```

Figure 6.54: Raw MongoDB Query for most used packages

6.2.5 Trending Packages Analysis (Last Month)

The `mostUsedPackagesLastMonth` method performs a time-sensitive frequency analysis on the `project` collection. Unlike the global analysis, this query isolates trends by considering only projects that have been updated within the last 30 days. This allows the administrator to identify currently trending libraries and emerging dependencies rather than historical standards.

The aggregation pipeline optimizes the process by filtering out dormant projects immediately. The process is structured as follows:



Figure 6.55: Illustration of the aggregation pipeline for recent package trends.

1. **Temporal Filtering (match):** Restricts the dataset to documents where the `last_update` field is greater than or equal to the calculated timestamp for "one month ago". Placing this filter at the start ensures that the expensive unwind operation is performed only on active projects.
2. **Deconstruction (unwind):** Flattens the `packages` array to handle each dependency as a discrete entity.

3. **Frequency Grouping (group):** Groups the dependencies by **name** and **version**, counting the occurrences within the active project set.
4. **Ranking (sort):** Orders the results by frequency in descending order to highlight the most popular recent choices.
5. **Top-K Selection (limit):** The result is limited to 10 elements to maintain focus on the top trends.
6. **Projection (project):** Formats the output to match the `AggreagationPackageDTO` structure, discarding internal identifiers.

Sample Result The result is returned as a list of `AggreagationPackageDTO` objects. The output structure mirrors the global analysis but reflects recent usage data:

```
[
  {
    "packageName": "axios",
    "version": "1.6.0",
    "count": 42
  },
  {
    "packageName": "moment",
    "version": "2.29.4",
    "count": 38
  }
]
```

Method Implementation The `mostUsedPackagesLastMonth` method implementation in Java is shown below, it's located in the `ProjectDAO` repository class.

```
1 public List<AggreagationPackageDTO> mostUsedPackagesLastMonth(int limit) {
2     LocalDateTime oneMonthAgo = LocalDateTime.now().minusMonths(1);
3
4     Aggregation aggregation = Aggregation.newAggregation(
5         Aggregation.match(Criteria.where("last_update").gte(oneMonthAgo)),
6
7         Aggregation.unwind("packages"),
8
9         Aggregation.group("packages.name", "packages.version").count().as("count"),
10
11        Aggregation.sort(Sort.Direction.DESC, "count"),
12
13        Aggregation.limit(limit),
14
15        Aggregation.project("count")
16            .and("_id.name").as("packageName")
17            .and("_id.version").as("version")
18            .andExclude("_id")
19    );
20
21    return mongoTemplate.aggregate(aggregation, "project", AggreagationPackageDTO.class)
22        .getMappedResults();
23 }
```

Figure 6.56: Method implementation of `mostUsedPackagesLastMonth`

MongoDB Shell Implementation The script below demonstrates the raw MongoDB query. It applies a temporal filter using `$dateSubtract`

```

1  [
2    {
3      "$match": { "$expr": { "$gte": [
4        "$last_update",
5        {
6          "$dateSubtract": {
7            "startDate": "$$NOW",
8            "unit": "month",
9            "amount": 1
10         }
11       }
12     ] } }
13   },
14   { "$unwind": "$packages" },
15   {
16     "$group": {
17       "_id": {
18         "name": "$packages.name",
19         "version": "$packages.version"
20       },
21       "count": { "$sum": 1 }
22     }
23   },
24   { "$sort": { "count": -1 } },
25   { "$limit": 10 },
26   {
27     "$project": {
28       "_id": 0,
29       "packageName": "$_id.name",
30       "version": "$_id.version",
31       "count": 1
32     }
33   }
34 ]

```

Figure 6.57: Raw MongoDB query of `mostUsedPackagesLastMonth`

6.2.6 Active Contributors Analysis

The `getTopCollaboratorsLastMonth` method executes a user-centric aggregation on the `project` collection. Its objective is to identify the most active contributors within the platform's ecosystem over the last 30 days. By analyzing the `collaborators` array in recently updated projects, the system highlights key users who are driving development activity.

The aggregation pipeline ensures performance by applying filters before deconstructive operations. The process is structured as follows:



Figure 6.58: Illustration of the aggregation pipeline for top contributors analysis.

1. **Temporal Filtering (match):** Filters the project pool to retain only those where the `last_update`

timestamp is within the last month. This efficient pre-filtering ensures that the subsequent array unwinding is performed only on relevant, active projects.

2. **Deconstruction (unwind):** Breaks down the `collaborators` array. Since a project involves multiple users, this step generates a separate document for each collaborator, allowing for individual contribution counting.
3. **Activity Grouping (group):** Groups the documents by the collaborator's `username`. It counts the number of distinct active projects each user is associated with.
4. **Reshaping (project):** Maps the result to the `ContributorCountDTO` schema. It explicitly renames the grouping identifier (`_id`) to `username` and retains the calculated `count`.
5. **Ranking (sort):** Sorts the contributors by their activity count in descending order.
6. **Top-K Selection (limit):** Limits the result set to the top 10 most active users to provide a concise leaderboard for the administrator.

Sample Result The result is returned as a list of `ContributorCountDTO` objects. The JSON output below illustrates the data structure:

```
[
  {
    "username": "francesca.romano",
    "count": 12
  },
  {
    "username": "daniel.martin",
    "count": 8
  }
]
```

Method Implementation The `getTopCollaboratorsLastMonth` method implementation in Java is shown below, it's located in the `ProjectDAO` repository class.

```
1 public List<ContributorCountDTO> getTopCollaboratorsLastMonth() {
2     LocalDateTime oneMonthAgo = LocalDateTime.now().minusMonths(1);
3
4     Aggregation aggregation = Aggregation.newAggregation(
5         Aggregation.match(Criteria.where("last_update").gte(oneMonthAgo)),
6
7         Aggregation.unwind("collaborators"),
8
9         Aggregation.group("collaborators.username").count().as("count"),
10
11         Aggregation.project("count")
12             .and("_id").as("username")
13             .andExclude("_id"),
14
15         Aggregation.sort(Sort.Direction.DESC, "count"),
16
17         Aggregation.limit(10)
18     );
19
20     return mongoTemplate.aggregate(aggregation, "project", ContributorCountDTO.class)
21         .getMappedResults();
22 }
```

Figure 6.59: Method implementation of `getTopCollaboratorsLastMonth`

MongoDB Shell Implementation The following script represents the raw MongoDB aggregation query corresponding to the Java logic. It demonstrates the direct filtering on the date field followed by the unwinding of the collaborators array.

```
1  [
2    {
3      "$match": { "$expr": { "$gte": [
4        "$last_update",
5        {
6          "$dateSubtract": {
7            "startDate": "$$NOW",
8            "unit": "month",
9            "amount": 1
10         }
11       }
12     ] } }
13   },
14   {
15     "$unwind": "$collaborators"
16   },
17   {
18     "$group": {
19       "_id": "$collaborators.username",
20       "count": { "$sum": 1 }
21     }
22   },
23   {
24     "$project": {
25       "_id": 0,
26       "username": "$_id",
27       "count": 1
28     }
29   },
30   {
31     "$sort": {
32       "count": -1
33     }
34   },
35   {
36     "$limit": 10
37   }
38 ]
```

Figure 6.60: Raw MongoDB Query for active contributors.

7 Database Architecture

The RiskDeV platform due to its dual-store architecture necessitates explicit trade-offs regarding consistency, availability, and partition tolerance across distributed infrastructure.

7.1 Distributed Data Strategy

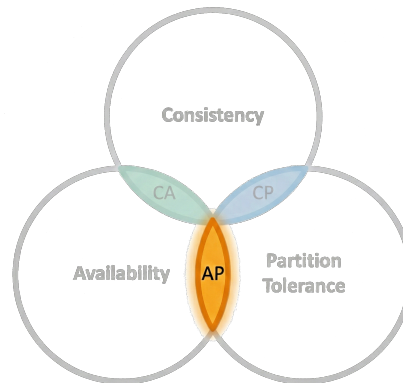


Figure 7.1: Availability and Partition Tolerance Choiche

CAP Theorem Positioning The system adopts an **AP (Availability and Partition Tolerance)** approach. The architecture prioritizes continuous write availability and fault tolerance across distributed infrastructure, accepting temporary divergence between data in MongoDB and Neo4j. This design is ideal for services requiring rapid and continuous responses, such as RiskDeV.

By decoupling write acknowledgment from cross-database synchronization, the system guarantees immediate persistence confirmation.

7.2 Mongo Replica Set

To satisfy the system's AP requirements, the document store is deployed as a **MongoDB Replica Set**. This configuration ensures high availability and fault tolerance by replicating data across multiple database instances while accepting temporary consistency divergence.

Primary–Secondary Architecture The **Primary** node is the exclusive entry point for write operations and records all changes in its operations log. **Secondary** nodes asynchronously pull and apply these log entries to converge toward the Primary's state.

By encapsulating replication, leader election, and failover within the replica set, MongoDB presents a unified logical database view to the application. This abstraction isolates database behavior from node failures and deployment topology, effectively decoupling the logical database state from the underlying physical infrastructure described in the subsequent section.

Replica Set Configuration The replica set topology and election preferences are explicitly defined during initialization. Each member is assigned a priority value, influencing primary election outcomes while preserving fault tolerance. The configuration used in this deployment is shown below:

```
rsconf = {
  _id: "lsmdb",
  members: [
    { _id: 0, host: "10.1.1.68:27017", priority: 1 },
    { _id: 1, host: "10.1.1.69:27017", priority: 2 },
    { _id: 2, host: "10.1.1.70:27017", priority: 5 }
  ]
}
```

Higher priority values bias the election process toward the designated Primary node while still allowing automatic failover in the presence of node failures.

Write Availability Guarantees Write availability is enforced through driver-level configuration. The write concern `w=1` and the journal setting `journal=true` acknowledges write operations immediately after the Primary persists data to memory and journal, without waiting for replication to Secondary nodes. This minimizes write latency and supports high-throughput ingestion workloads. Additionally, `retryWrites=true` enables the driver to automatically retry transiently failed write operations, masking short-lived network partitions and improving write robustness.

Read Distribution Strategy Read operations are governed by the `readPreference=nearest` setting, which directs queries to the replica set member with the lowest network latency. In the deployed topology—where the application is co-located with a MongoDB Secondary—this effectively results in local read access.

This strategy explicitly trades strong consistency for reduced latency, tolerating limited staleness in exchange for faster query responses. Such a trade-off aligns with the AP model and is suitable for latency-sensitive services such as RiskDeV.

7.3 Infrastructure Topology

The system is deployed across three VMs, distributing database and application workloads to satisfy high-availability and performance requirements defined in previous sections. Each node hosts a specific subset of services as reported in Figure 7.2.

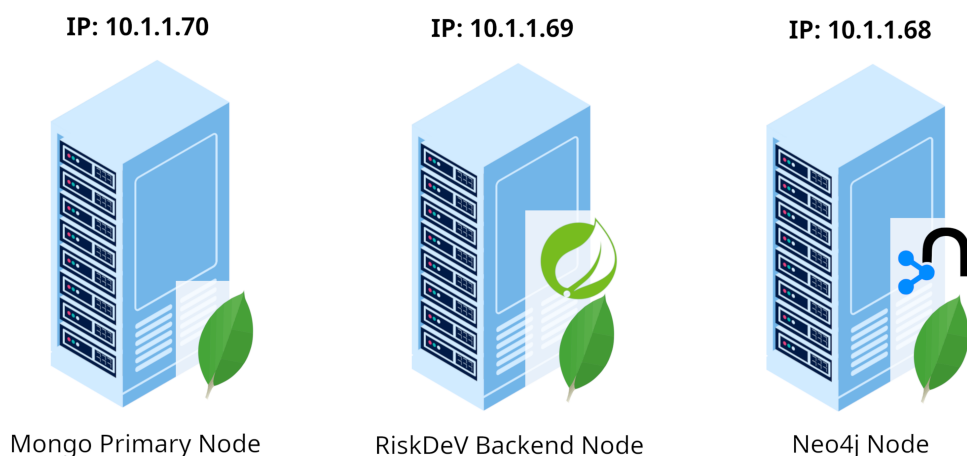


Figure 7.2: Deploy Configuration

The cluster follows a clear separation-of-responsibilities model, where each node is assigned a well-defined role within the system:

- **Mongo Primary Node:** This node is dedicated exclusively to the MongoDB **Primary** instance.
- **RiskDeV Backend Node:** This node hosts the Spring Boot application alongside a MongoDB **Secondary** instance.
- **Neo4j Node:** This node hosts Neo4j together with a MongoDB **Secondary** instance.

This distribution isolates the write-intensive Primary from read-heavy application traffic and graph-processing workloads, improving overall system stability.

7.4 Eventual Consistency Across MongoDB and Neo4j

The system maintains **eventual consistency** between MongoDB and Neo4j through an event-driven architecture introduced in Chapter 5. In this chapter, only an example is shown to clarify its behavior. Writes committed to MongoDB are acknowledged immediately, while asynchronous events propagate updates to Neo4j. This introduces a small window in which clients querying the graph immediately after a document update may encounter stale data until propagation completes.

The discussion below uses the release of a new package version as a reference for system insertion. After the insertion request, the admin immediately receives a response from the system. The write of the new release is synchronously written only in MongoDB (Figure 7.3). If the operation fails, a dedicated **Executor** handles the exception and returns an error message to the admin (Figure 7.4). Otherwise, the system publishes an appropriate event.

```

1 public Result<MessageResponseDTO> addNewVersion(String packageName, AddPackageVersionDTO newVersionDTO) {
2     if (!packageName.equals(newVersionDTO.getPackageName())) {
3         return new Result.Failure<>(new DomainError.InvalidOperation("The package name in the URL does not match the body"));
4     }
5
6     String version = newVersionDTO.getVersion();
7     if (packageVersionRepository.existsByPackageNameAndVersion(packageName, version)) {
8         return new Result.Failure<>(new DomainError.AlreadyExists("Version " + version + " already exists."));
9     }
10
11     PackageVersion versionDoc = new PackageVersion(newVersionDTO, versionParser);
12     packageVersionRepository.save(versionDoc);
13
14     PublishedVersionDTO publishedVersionDTO = new PublishedVersionDTO(versionDoc);
15     eventPublisher.publishEvent(new PackageEvent.VersionRelease(publishedVersionDTO));
16
17     return new Result.Success<>(new MessageResponseDTO("Package version created successfully"));
18 }

```

Figure 7.3: Package version insertion logic

```

1 package it.unipi.riskDeV.util;
2 import java.util.concurrent.Callable;
3
4 import it.unipi.riskDeV.results.DomainError;
5 import it.unipi.riskDeV.results.Result;
6 import lombok.extern.slf4j.Slf4j;
7
8 @Slf4j
9 public class ResultExecutor {
10     public static <T> Result<T> execute(Callable<Result<T>> action) {
11         try {
12             return action.call();
13         }
14         catch (Exception e) {
15             log.error("Unhandled exception caught in executor", e);
16             return new Result.Failure<>(new DomainError.SystemError(e));
17         }
18     }
19 }
20

```

Figure 7.4: Executor

The published event is captured by a **listener**, which asynchronously synchronizes Neo4j (Figure 7.5). If this operation fails, the listener saves a **FailedEvent** in a dedicated **DLQ** stored in MongoDB using the **failedEventService** (Figure 7.6).

```

1  @Async
2  @EventListener
3  public void handlePackageEvent(PackageEvent event) {
4      log.debug("[Graph Listener] Processing package event: {}", event.packageName());
5
6      try {
7          switch (event) {
8              case PackageEvent.VersionRelease v →
9                  handleVersionRelease(v);
10
11             case PackageEvent.UpdateDocumentation d →
12                 graphService.updatePackageDocumentation(d.packageName(), d.documentationURL());
13
14             case PackageEvent.UpdatePackageVersion u →
15                 graphService.updatePackageVersion(u.packageName(), u.version(), u.dependencies(), u.vulnerabilities());
16
17             case PackageEvent.DeletePackageVersion del →
18                 graphService.deletePackageVersion(del.packageName(), del.version());
19             }
20         } catch (Exception e) {
21             failedEventService.saveError(event, "Neo4j Package Sync Error: " + e.getMessage());
22         }
23     }
24
25     private void handleVersionRelease(PackageEvent.VersionRelease v) {
26         Double riskScore = helper.getMaxBaseScore(v.publishedVersionDTO().getVulnerabilities());
27
28         documentService.updateRiskScore(v.publishedVersionDTO().getPackageName(), v.publishedVersionDTO().getVersion(), riskScore);
29         graphService.addPackage(v.publishedVersionDTO(), riskScore);
30     }

```

Figure 7.5: Package event handler logic in the listener

```

1  @Service
2  @RequiredArgsConstructor
3  @Slf4j
4  public class FailedEventService {
5
6      private final FailedEventRepository failedEventRepository;
7      private final ObjectMapper objectMapper;
8
9      public void saveError(Object event, String errorMessage) {
10         try {
11             FailedEvent dlq = new FailedEvent();
12             dlq.setEventType(event.getClass().getSimpleName());
13             dlq.setExceptionMessage(errorMessage);
14             dlq.setFailedAt(Instant.now());
15             dlq.setRetryCount(0);
16
17             try {
18                 String jsonPayload = objectMapper.writeValueAsString(event);
19                 dlq.setPayloadJson(jsonPayload);
20             } catch (Exception jsonEx) {
21                 log.error("Failed to serialize event payload to JSON", jsonEx);
22                 dlq.setPayloadJson("Serialization Failed: " + event.toString());
23             }
24
25             failedEventRepository.save(dlq);
26
27             log.warn("[DLQ] Event {} saved to DLQ. Reason: {}", event.getClass().getSimpleName(), errorMessage);
28
29         } catch (Exception e) {
30             log.error("CRITICAL: Failed to save event to DLQ. Original Error: {}. DLQ Error: {}", errorMessage, e.getMessage(), e);
31         }
32     }
33 }

```

Figure 7.6: Failed event service

A dedicated scheduler, the **DlqRetryScheduler**, retries failed events from the DLQ every 5 minutes for a maximum of 5 attempts, restoring consistency when needed (Figure 7.7). This component uses a strategy pattern, with all strategies defined in a dedicated handler — in this example, the **packageEventHandler** (Figure 7.8). The overall event-driven consistency strategy, including the interaction between MongoDB,

the listener, the DLQ, and Neo4j, is summarized in Figure 7.9.

```

1  @Component
2  @RequiredArgsConstructor
3  @Slf4j
4  public class DLQRetryScheduler {
5
6      private final FailedEventRepository failedEventRepository;
7      private final List<EventHandler> eventHandlers;
8      private static final int BATCH_SIZE = 50;
9      private static final int MAX_RETRIES = 5;
10
11      @Scheduled(fixedDelayString = "${app.scheduler.retry.delay:300000}")
12      public void retryFailedEvents() {
13
14          Pageable limit = PageRequest.of(0, BATCH_SIZE);
15          List<FailedEvent> events = failedEventRepository.findByResolvedAtIsNullAndRetryCountLessThan(MAX_RETRIES, limit);
16
17          if (events.isEmpty()) return;
18
19          log.info("[DLQ Scheduler] Processing batch of {} failed events.", events.size());
20
21          for (FailedEvent failedEvent : events) {
22              processSingleEvent(failedEvent);
23          }
24      }
25
26      private void processSingleEvent(FailedEvent failedEvent) {
27          try {
28              EventHandler handler = eventHandlers.stream()
29                  .filter(h → h.canHandle(failedEvent.getEventType()))
30                  .findFirst()
31                  .orElseThrow(() → new IllegalArgumentException("No handler found for type: " + failedEvent.getEventType()));
32
33              handler.handle(failedEvent.getPayloadJson());
34              markAsResolved(failedEvent);
35
36          } catch (Exception e) {
37              log.warn("Retry failed for event {}: {}", failedEvent.getId(), e.getMessage());
38              markAsFailedAgain(failedEvent, e.getMessage());
39          }
40      }
41  }
42

```

Figure 7.7: Scheduler

```

1  @Service
2  @RequiredArgsConstructor
3  @Slf4j
4  public class PackageEventHandler implements EventHandler {
5
6      private final GraphService graphService;
7      private final DocumentService documentService;
8      private final ObjectMapper objectMapper;
9      private final Helper helper;
10
11      @Override
12      public boolean canHandle(String eventType) {
13          return eventType.contains("Package") || eventType.contains("Version") || eventType.contains("Documentation");
14      }
15
16      @Override
17      public void handle(String payloadJson) {
18          PackageEvent event;
19          try {
20              event = objectMapper.readValue(payloadJson, PackageEvent.class);
21          } catch (Exception e) {
22              throw new RuntimeException("JSON Deserialization failed", e);
23          }
24
25          log.debug("DLQ Retry: Package event for {}", event.packageName());
26
27          switch (event) {
28              case PackageEvent.VersionRelease v → {
29                  Double riskScore = helper.getMaxBaseScore(v.publishedVersionDTO().getVulnerabilities());
30
31                  documentService.updateRiskScore(v.publishedVersionDTO().getPackageName(), v.publishedVersionDTO().getVersion(), riskScore);
32                  graphService.addPackage(v.publishedVersionDTO(), riskScore);
33              }
34
35              case PackageEvent.UpdateDocumentation d →
36                  graphService.updatePackageDocumentation(d.packageName(), d.documentationURL());
37
38              case PackageEvent.UpdatePackageVersion u →
39                  graphService.updatePackageVersion(u.packageName(), u.version(), u.dependencies(), u.vulnerabilities());
40
41              case PackageEvent.DeletePackageVersion del →
42                  graphService.deletePackageVersion(del.packageName(), del.version());
43          }
44      }
45  }

```

Figure 7.8: Package Event Handler

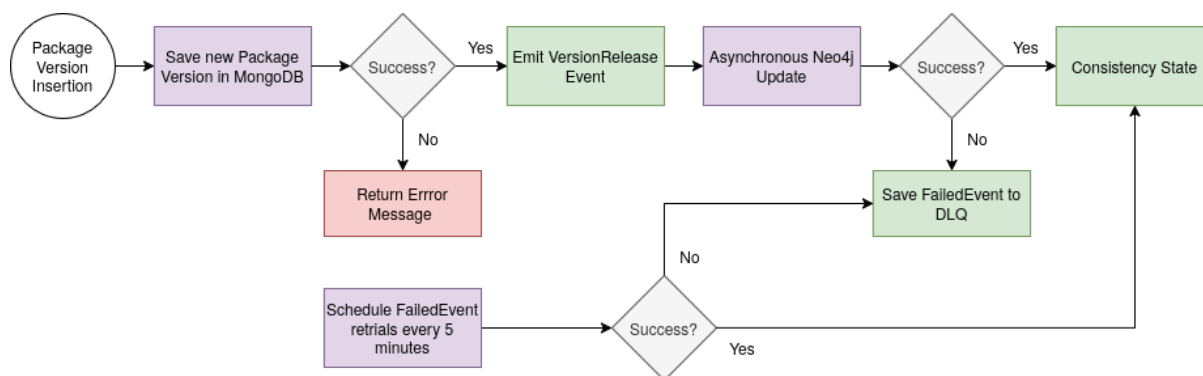


Figure 7.9: Consistency Strategy Scheme

7.5 Sharding Strategy

MongoDB achieves horizontal scaling by distributing data across multiple shards. Selecting a **shard key** is the most critical design decision, as it dictates write distribution and query efficiency. While MongoDB 4.2+ supports shard key updates, keys should remain **stable** to avoid the heavy I/O overhead of document migrations.

7.5.1 Proposed Sharding Design

While the system currently operates on a single Replica Set, it is designed to be sharding-ready. The following strategies describe how collections **could be sharded** to support horizontal growth:

- **Project Collection:** *Hashed* sharding on the **name**. Another choiche could be a *Range-based* sharding based on the **name** but the distribution of the name isn't fair and could produce an high difference between different collections size. However agregations performance are acceptable as these are infrequent, low-priority admin operations.
- **Package Collection:** Sharding on **package_name**. *Range-based* sharding favors prefix searches but risks hotspots; *Hashed* sharding is preferred for high-concurrency point lookups to ensure writes load balancing.
- **Vulnerability Collection:** *Hashed* Sharding on **cve_id** is required to mitigate a strong inequality between shards cardinalities. Since CVEs are issued sequentially, range-based sharding would bottleneck all writes on the "latest" shard.
- **User Collection:** *Hashed* sharding on **username** is prefereable allowing collections to be filled evenly, in addition the system does not involve aggregation on this collection.
- **DLQ Collection:** *Hashed* Sharding on **_id**. This prevents the collection, in network partition situation, from becoming a bottleneck during error bursts by parallelizing ingestion across all shard primaries.

8 System Performance Evaluation

This chapter presents an experimental evaluation of the system's performance, focusing on the impact of database design and deployment choices on query efficiency and scalability. The analysis is structured around two main aspects: the effect of indexing strategies in both MongoDB and Neo4j, and the behavior of the system under different MongoDB read preference configurations. Through execution plan comparisons and controlled load tests, this chapter highlights how these design decisions influence response time, resource utilization, and the system's ability to handle increasing levels of concurrency.

8.1 Indexes

This section analyzes the impact of database indexes on the performance of the system. Indexes have been created in both MongoDB and Neo4j to improve query efficiency. By reducing the amount of data scanned during query execution, indexes allow the database engine to locate relevant documents or nodes more quickly. As a result, response times are significantly improved, particularly for frequent search and aggregation operations across the system.

8.1.1 MongoDB

In MongoDB, indexes were created across all the collections to improve the performance of the most frequent queries, including searches, retrievals of specific documents and aggregation operations. Some indexes were defined as unique to enforce data consistency, providing a double layer of validation: one at the application level in the code and another at the database level through the unique constraint. This ensures that documents cannot be duplicated on key fields, while also enabling faster query execution by allowing the database to efficiently locate relevant records.

Collection – Package

The **Package** collection stores all package metadata and version information, making it one of the most frequently accessed collections in the system. To optimize query performance, several indexes were created:

- A single-field ascending index on **package_name** to speed up lookups by package name.
- A unique compound ascending index on **package_name** and **version** to efficiently retrieve a specific package version.
- A compound index on **package_name** (ascending) and **version_array** (descending) to optimize queries involving version ranges or aggregations across multiple versions.

The following figures compare the execution plans and response times of key queries before and after the introduction of these indexes. The results show a clear reduction in the number of scanned documents and a significant improvement in overall query performance, confirming that proper indexing is essential for system scalability and responsiveness.



Figure 8.1: Execution plan and response time for `getPackageByName` without an index on the package name field



Figure 8.2: Execution plan and response time for `getPackageByName` with an index on the package name field



Figure 8.3: Execution plan and response time for `getPackageVersion` without a compound index on package name and version



Figure 8.4: Execution plan and response time for `getPackageVersion` with a compound index on package name and version

Collection – Project

The **Project** collection stores user-defined project configurations and supports both direct lookup queries and time-based aggregations. To improve query efficiency, two secondary indexes were introduced:

- A unique index on the `name` field to accelerate direct project retrieval and enforce name uniqueness.
- A descending index on the `last_update` field to optimize temporal aggregations over recent project activity.

These indexes significantly reduce full collection scans and enable the query planner to leverage index-based filtering and sorting, resulting in lower execution time and improved scalability.



Figure 8.5: Execution plan and response time for `getProject` without an index on the `name` field



Figure 8.6: Execution plan and response time for `getProject` with a unique index on the `name` field



Figure 8.7: Aggregation performance for `mostUsedPackagesLastMonth` without an index on the `last_update` field



Figure 8.8: Aggregation performance for `mostUsedPackagesLastMonth` with a descending index on the `last_update` field

Collection – User

The `User` collection stores authentication and profile data for registered users and is frequently accessed during login, authorization and administrative operations. To ensure fast and reliable user lookup, a unique secondary index is defined on the `username` field. This index both enforces uniqueness constraints and enables constant-time retrieval for queries that resolve user identities, significantly reducing query latency and avoiding full collection scans.



Figure 8.9: Execution plan and response time for `getUser` without a supporting index on the `username` field



Figure 8.10: Execution plan and response time for `getUser` with a unique index on the `username` field

Collection – Vulnerability

The `Vulnerability` collection stores detailed CVE records, including publication dates and CVSS security metrics, and supports both direct lookups and analytical queries. To optimize these access patterns, multiple indexes are defined:

- A unique index on the `cve_id` field enables fast retrieval of individual vulnerabilities and enforces uniqueness across the dataset.
- An index on the `published` field improves the performance of time-based queries, such as trend analysis over recent periods.
- a compound index on key CVSS metrics (`accessVector`, `authentication`, `accessComplexity`, `baseScore`, and `baseSeverity`) supports advanced filtering and ranking operations used to identify the most critical and high-risk vulnerabilities.



Figure 8.11: Execution plan and response time for `getVulnerabilityByCVE` without a supporting index on the `cve_id` field



Figure 8.12: Execution plan and response time for `getVulnerabilityByCVE` with a unique index on the `cve_id` field



Figure 8.13: Aggregation performance for `getTrendVulnerabilityLastYear` without an index on the `published` field

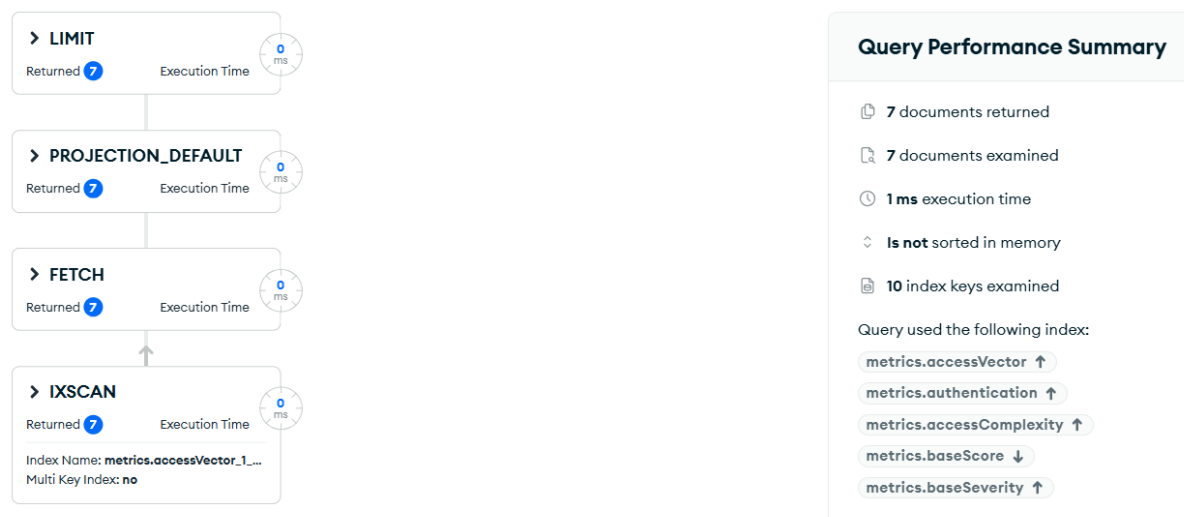


Figure 8.14: Aggregation performance for `getTrendVulnerabilityLastYear` with an index on the `published` field



Figure 8.15: Aggregation performance for `getMostDangerousVulnerabilities` without a compound index on CVSS metric fields



Figure 8.16: Aggregation performance for `getMostDangerousVulnerabilities` with a compound index on CVSS metric fields

Collection – DLQ

In this collection, we set a TTL index on the `resolvedAt` field. Since this collection contains asynchronous events that failed to be processed by Neo4j, once they are resolved the associated document is updated. However, there is no need to retain documents related to resolved events indefinitely, so we configured them to be automatically deleted after 7 days.

8.1.2 Neo4j

In the Neo4j graph database, both constraints and indexes are employed to guarantee data consistency and to significantly enhance query performance. Uniqueness constraints are defined on `Vulnerability.cve_id`, `Package.package_name`, and `Project.name`, ensuring that each entity is stored only once and can be reliably retrieved. In addition, a uniqueness constraint is defined on the pair (`Version.package_name`, `Version.version`), replacing the previous compound index on the same fields. This constraint not only enables fast indexed lookups but also prevents duplicate version nodes from being inserted.

Beyond the uniqueness constraints, a separate single-field index on `Version.package_name` is maintained to optimize queries that search for all versions of a package without specifying the version string. This ensures that even partial queries on versions benefit from efficient index access.

The following execution plans compare database behavior before and after the introduction of these constraints and indexes. In all cases, indexed execution paths show a clear reduction in the number of database hits, demonstrating more efficient access patterns and improved query performance.

The first comparison illustrates the retrieval of a vulnerability node by its CVE identifier. Without a uniqueness constraint, Neo4j performs a full scan over all `Vulnerability` nodes. With the constraint in place, the query is resolved through a direct indexed lookup, significantly reducing the number of accessed nodes.

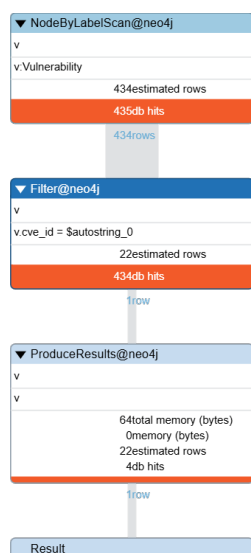


Figure 8.17: Retrieving a vulnerability by CVE identifier without an index, resulting in a full node scan

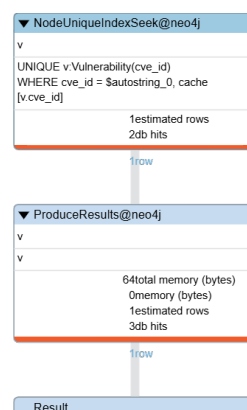


Figure 8.18: Retrieving a vulnerability by CVE identifier using a uniqueness constraint, enabling an indexed lookup

The second comparison illustrates the retrieval of a package node by its name. In the absence of an index, Neo4j must scan all `Package` nodes in the graph. After enabling the uniqueness constraint on the `package_name` property, the query is resolved through a direct index seek, leading to fewer database hits and faster execution.

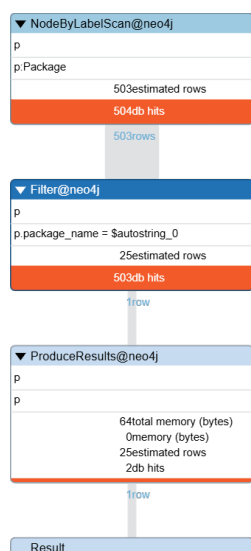


Figure 8.19: Retrieving a package by name without an index, requiring a scan over all package nodes

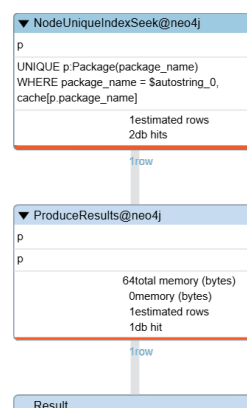


Figure 8.20: Retrieving a package by name using a uniqueness constraint, enabling a direct index seek

The third comparison focuses on retrieving a project node by its name. Without a supporting constraint, Neo4j performs a full scan of all **Project** nodes. When the uniqueness constraint is applied, the database directly locates the matching node through the index, minimizing unnecessary graph traversal.

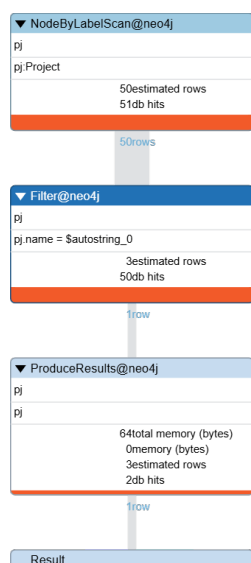


Figure 8.21: Retrieving a project by name without an index, resulting in a full scan of project nodes

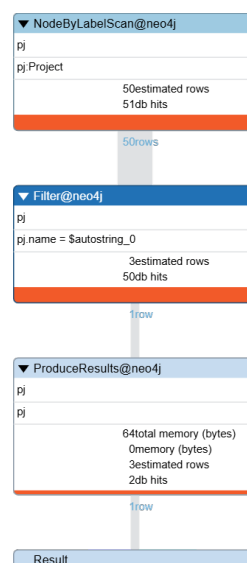


Figure 8.22: Retrieving a project by name using a uniqueness constraint, enabling an indexed lookup

The final comparisons analyze queries on version nodes. The first case retrieves a specific version of a package using both the package name and the version string, while the second case retrieves all versions associated with a given package. Without dedicated indexes, both operations require scanning a large portion of the **Version** nodes. With the compound and single-field indexes enabled, Neo4j performs efficient index seeks, consistently reducing the number of database hits and improving execution time.

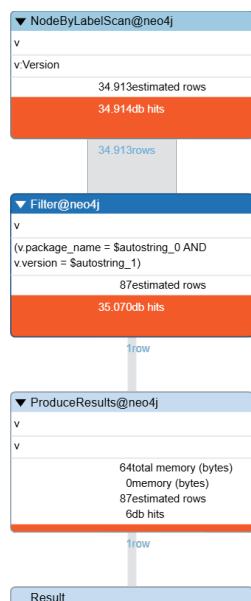


Figure 8.23: Retrieving a specific package version without a compound index, requiring a scan over version nodes

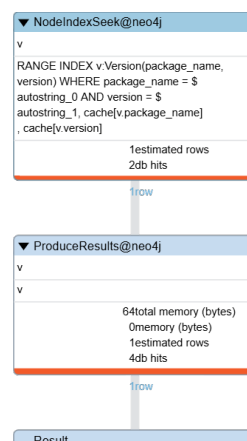


Figure 8.24: Retrieving a specific package version using a compound index on package name and version

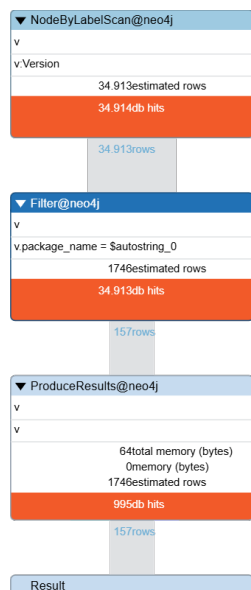


Figure 8.25: Retrieving all versions of a package without an index, resulting in a large number of scanned nodes

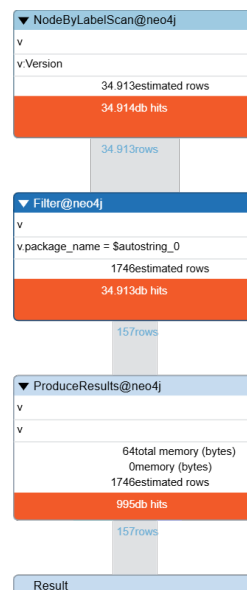


Figure 8.26: Retrieving all versions of a package using an index on the package name, enabling efficient filtering

8.2 Read Preference Performance Comparison

This section evaluates the impact of different MongoDB read preferences on system performance, comparing `readPreference=primary` and `readPreference=nearest`. These configurations control how the client selects a node in the replica set to serve read operations and influence both latency and scalability under load.

With `readPreference=primary`, all read requests are routed to the primary node. This ensures strong consistency, as clients always read the most recent committed data. However, this approach centralizes

both read and write workloads on a single node, which can limit scalability as the number of concurrent requests increases.

In contrast, `readPreference=nearest` allows read requests to be served by the node with the lowest network latency, which can be either the primary or a secondary. This enables read traffic to be distributed across multiple members of the replica set. While this may introduce eventual consistency for some reads, it reduces pressure on the primary node and improves the system's ability to scale under higher read loads.

Load tests were conducted using Postman with a peak load profile, simulating 100 virtual users over a 5-minute test duration and a base load of 50 concurrent users. The same set of API endpoints and request patterns was applied in both configurations to ensure a consistent comparison.

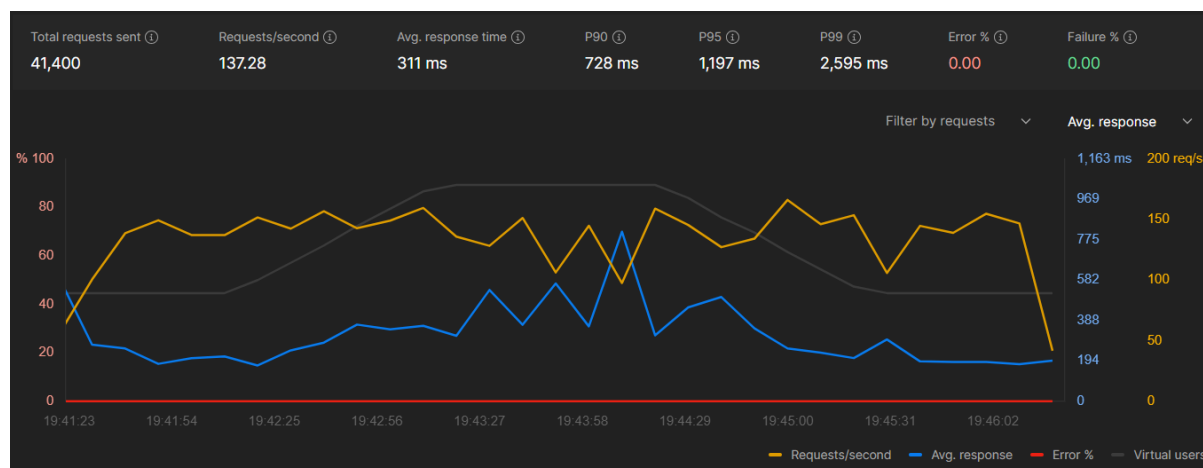


Figure 8.27: Performance metrics with `readPreference=nearest` (Average response time: 311 ms)

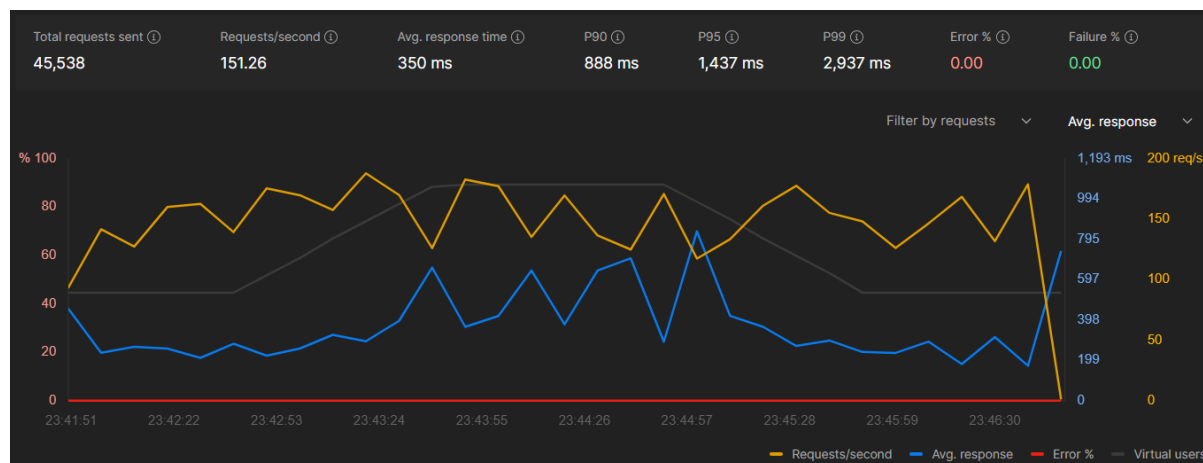


Figure 8.28: Performance metrics with `readPreference=primary` (Average response time: 350 ms)

The measured results indicate that `readPreference=nearest` provides slightly better latency characteristics in this test scenario, although the difference is not significant. Average and tail latencies (P90 and P95) are marginally lower when reads are allowed to be served by secondary nodes as well as the primary. Throughput remains comparable between the two configurations.

From a conceptual and architectural perspective, these results support the idea that, as the volume of read requests increases, a replica set configuration where secondary nodes can also serve reads offers better scalability. By distributing read operations across multiple nodes instead of concentrating them solely on the primary, the system can handle higher levels of concurrency more efficiently, even if the immediate performance gains in small-scale tests are limited.

9 Use of Generative AI Tools

During the development and documentation phases of the RiskDev project, Large Language Models (LLMs), specifically OpenAI's ChatGPT and Google Gemini, were utilized as supportive tools to enhance productivity and ensure technical precision.

The application of these tools focused on:

- **Frontend Prototyping (Mockups):** Google Gemini was employed to generate the HTML and CSS code for the Guest, User and Administrator mockups. The prompting process involved providing detailed descriptions of the desired layout and mapping specific data fields retrieved from our backend queries to UI elements.
- **Documentation and Translation:** Both ChatGPT and Gemini were used to draft and refine technical documentation. They supported the translation of concepts into precise technical English, helping to eliminate typographical errors and reduce linguistic redundancy.
- **LaTeX Typesetting:** The integrated AI assistant within the Prism (formerly Crixet) editor was leveraged to automate the formatting of complex LaTeX elements, specifically for the configuration of tables and image alignment, streamlining the final typesetting process.
- **Technical Research and Pattern Analysis:** Gemini were utilized to explore advanced Java constructs and architectural patterns. All AI-generated technical suggestions were subsequently verified against official documentation and industry standards to ensure code robustness and architectural integrity.